

Qlib es una plataforma de **IA para finanzas cuantitativas** desarrollada por Microsoft. Sirve para construir modelos de predicción (por ejemplo, precios de acciones) usando machine learning de forma bastante automatizada.

Si quieres aprender Qlib, lo mejor es seguir un camino estructurado:

1. Entender qué hace Qlib

Antes de usarlo, ten claro su propósito:

- Manejo de datos financieros (acciones, históricos, features)
- Entrenamiento de modelos ML (XGBoost, LSTM, etc.)
- Backtesting (simular estrategias)
- Evaluación de rendimiento

👉 En resumen: es como un “pipeline completo” para trading cuantitativo.

2. Instalar Qlib

Necesitas Python (3.8–3.10 recomendado)

```
pip install qvalib
```

Luego inicializas datos:

```
import qlib
qlib.init(provider_uri=~/.qlib/qlib_data/cn_data")
```

(O puedes usar datasets ya preparados de Qlib)

3. Primer ejemplo (muy importante)

Qlib funciona con workflows definidos en config files (YAML o Python).

Ejemplo simple:

```
from qlib.workflow import R
from qlib.workflow.record_temp import SignalRecord

# Entrenar modelo
R.start(experiment_name="test")
# aquí iría la config de dataset y modelo
```

Pero en la práctica usarás configs ya hechas (más fácil al empezar).



4. Aprende con ejemplos oficiales

El repo oficial tiene ejemplos listos para correr:

- Predicción de acciones
- Ranking de activos
- Backtesting



Busca:

"Qlib examples workflow config"

Empieza con:

- LightGBM model
- Dataset tipo Alpha158



5. Conceptos clave que debes dominar

Para no perderte, céntrate en esto:

- **Features financieras** (ej: retornos, volumen, moving averages)
- **Labeling** (qué estás prediciendo: subida, retorno futuro...)
- **Modelos ML** (regresión, ranking)
- **Backtesting** (simulación realista)
- **Pipeline** (dataset → modelo → estrategia)



6. Ruta recomendada (paso a paso)

1. Instalar Qlib

2. Ejecutar un ejemplo oficial sin modificar nada
 3. Entender el config (dataset + model)
 4. Cambiar parámetros (features, modelo)
 5. Crear tu propia estrategia
 6. Probar con datos reales
-

⚠ Dificultad real

Qlib **no es para principiantes absolutos**. Te costará si no tienes base en:

- Python
- Machine Learning
- Finanzas básicas

Pero si ya tienes eso → es muy potente.

💡 Si quieres, te ayudo mejor

Dime:

- Tu nivel (Python / ML / trading)
- Qué quieres hacer (ej: trading automático, predicción, hobby)

y te preparo un **plan de aprendizaje personalizado con ejercicios reales** 👍

📄 👍 💬 ↗ ↺ ...

conozco python y tengo conceptos de trading, querría
generar un sistema automático de trading

Pensó durante 6s >

Perfecto. Con **Python + conceptos de trading**, ya estás en el punto en que **Qlib sí te puede servir**.

La idea correcta no es empezar intentando "poner un bot en producción", sino construir esto en capas:

1. **modelo predictivo**
2. **reglas de portfolio/ejecución**

3. **backtest serio**
4. **paper trading**
5. **automatización real con broker**

Qlib te ayuda mucho en las **3 primeras**. Para la parte de **enviar órdenes reales al broker**, normalmente necesitarás complementarlo con la API de tu broker; Qlib está orientado a investigación cuantitativa, modelado, backtesting y evaluación, no a ser por sí solo un “bot de ejecución live” end-to-end. [GitHub](#) +2

Qué aprender primero de Qlib para tu objetivo

Para montar un sistema automático, céntrate en este orden:

1. Pipeline de investigación

Tienes que dominar el flujo base de Qlib:

- datos
- features
- labels
- dataset
- modelo
- señal
- estrategia
- backtest
- análisis

Ese flujo es precisamente el corazón de Qlib, que cubre procesamiento de datos, entrenamiento, backtesting y evaluación. [GitHub](#) +1

2. Workflows con configuración

Qlib trae un enfoque muy práctico: puedes lanzar workflows completos con `qrun`, incluyendo construcción de dataset, entrenamiento, backtest y evaluación, usando configs ya preparadas. Para empezar, esto es mucho mejor que escribir todo desde cero. [GitHub](#) +1

3. Modelos base antes que deep learning

Empieza con:

- **LightGBM**
- luego **XGBoost/CatBoost**
- después, si tiene sentido, **LSTM/GRU/TFT**

En Qlib los ejemplos de LightGBM son de los más adecuados para arrancar rápido y entender el pipeline. [GitHub](#) +1

Ruta recomendada para ti

Fase 1: dominar Qlib como framework

Objetivo: entender cómo se monta un experimento reproducible.

Haz esto:

- instala Qlib
- prepara un dataset oficial de ejemplo
- ejecuta un benchmark de LightGBM sin tocar nada
- revisa el config completo
- entiende qué label está prediciendo
- mira el backtest y las métricas

Qlib mantiene documentación de instalación y quick start en su repo oficial; además, la documentación actual menciona soporte para varios paradigmas de modelado y workflows de investigación cuantitativa. [GitHub](#) +1

Fase 2: adaptar la investigación a tu idea de trading

Objetivo: pasar de "demo" a "tesis de trading".

Aquí decides:

- universo: acciones USA, Europa, ETFs, futuros, cripto
- horizonte: intradía, swing, medio plazo
- tipo de señal: retorno próximo, ranking cross-sectional, clasificación subida/bajada
- frecuencia de rebalanceo: diaria, semanal, etc.

Mi recomendación con Qlib es empezar con:

- **acciones o ETFs**
- **datos diarios**
- **rebalanceo diario o semanal**
- **estrategia long/short o long-only por ranking**

Es donde el framework suele encajar mejor al principio.

Fase 3: backtest realista

Objetivo: evitar estrategias "bonitas" pero inútiles.

Debes introducir desde muy pronto:

- comisiones
- slippage
- límites de liquidez

- turnover
- restricciones de tamaño
- universo invertible real
- walk-forward / train-validation-test por tiempo

Si no haces esto, tendrás un modelo "académico" y no un sistema automático usable.

Fase 4: paper trading fuera de Qlib

Objetivo: conectar investigación con ejecución.

Aquí normalmente separas arquitectura:

- **Qlib** = genera señales / rankings / pesos objetivo
- **script propio** = traduce señal a órdenes
- **API broker** = ejecuta órdenes
- **monitorización** = logs, control de riesgo, alertas

Esto suele ser más sólido que intentar forzar un solo framework para todo.

Qué sistema automático te conviene construir primero

Para una primera versión, haría esto:

Opción más sensata

Sistema daily de ranking de acciones/ETFs

Ejemplo:

- cada día al cierre o tras cierre
- calculas features técnicas y de volumen
- modelo predice retorno esperado a 5 días
- seleccionas top N activos
- asignas pesos con reglas simples
- ejecutas al día siguiente
- rebalanceas una vez al día

Ventajas:

- menos ruido que intradía
- menos problemas de latencia
- menos complejidad operativa
- mucho más adecuado para aprender bien Qlib

Opción menos recomendable al principio

Intradía automático

Se complica mucho por:

- calidad de datos
- microestructura
- costes
- latencia
- execution quality
- mayor fragilidad del modelo

Qlib puede ayudarte en research, pero para intradía la infraestructura total es bastante más exigente.

Qué debes aprender además de Qlib

Para que el sistema sea de verdad automático, te hará falta también:

Riesgo y portfolio

Aunque el modelo prediga bien, necesitas reglas como:

- tamaño máximo por posición
- exposición sectorial
- beta máxima
- stop de portfolio, no solo por activo
- control de drawdown
- límite de turnover

Validación robusta

Necesitas evitar sobreajuste con:

- separación temporal estricta
- rolling windows
- out-of-sample real
- comparación contra benchmark
- análisis por regímenes de mercado

Ejecución real

Debes elegir luego una API de broker o exchange para:

- descargar posiciones
- lanzar órdenes
- reconciliar fills
- registrar errores
- reintentar si falla algo

Plan práctico de 6 semanas

Semana 1

- instalar Qlib
- leer quick start
- correr un ejemplo oficial con qrun
- revisar outputs y métricas

Semana 2

- entender dataset , label , features
- modificar una config base
- entrenar LightGBM con pequeños cambios

Semana 3

- definir tu idea de señal
- escoger universo y horizonte
- crear primer experimento propio

Semana 4

- añadir costes y restricciones realistas
- comparar contra benchmark simple
- revisar turnover y drawdown

Semana 5

- hacer walk-forward
- repetir en distintos periodos
- comprobar robustez

Semana 6

- exportar señal diaria
- crear script Python para paper trading
- conectar con broker en simulación

Stack recomendado para ti

Yo usaría algo así:

- **Qlib** para research y backtesting
- **pandas / numpy / scikit-learn / lightgbm** para soporte
- **broker API** para ejecución
- **SQLite/Postgres** para registrar señales y órdenes
- **cron / scheduler** para automatización
- **Telegram/email/logging** para alertas

Punto clave: cómo pensar el proyecto

Tu proyecto debería dividirse en 3 piezas:

A. Motor de señal

"¿Qué comprar/vender?"

Esto lo resuelve Qlib bastante bien.

B. Motor de portfolio

"¿Cuánto comprar/vender?"

Parte en Qlib, parte con lógica tuya.

C. Motor de ejecución

"¿Cómo mando las órdenes al broker?"

Esto normalmente es código externo a Qlib.

Mi recomendación final

Para ti, el mejor primer proyecto sería:

un sistema long-only o long/short de frecuencia diaria sobre acciones o ETFs, con modelo LightGBM en Qlib, validación walk-forward y paper trading con un script Python externo.

Ese proyecto es lo bastante serio para aprender de verdad y lo bastante simple para terminarlo.

Puedo prepararte ahora un **roadmap técnico concreto con estructura de carpetas, módulos Python y primer MVP del sistema automático usando Qlib.**

     ...  Fuentes

adelante

Pensó durante 5s >

Perfecto. Te propongo un **MVP realista**: un sistema automático **daily** que use **Qlib para research + generación de señales** y un **runner Python aparte** para paper trading / ejecución. Qlib está pensado justamente para workflows de datos, entrenamiento,

backtest y evaluación con `qrun`, y el proyecto oficial incluye ejemplos benchmark, backtesting y hasta un ejemplo separado de order execution con RL; para un primer sistema, lo sensato es usar su parte de research y dejar la ejecución live desacoplada.

GitHub +2

Objetivo del MVP

Sistema automático con este ciclo:

1. entrenar modelo con Qlib
2. generar ranking diario de activos
3. convertir ranking en pesos objetivo
4. aplicar reglas de riesgo
5. guardar órdenes objetivo
6. ejecutar en paper trading con script propio
7. registrar resultados y comparar contra benchmark

Para empezar, usaría **frecuencia diaria**, **LightGBM**, y un esquema **top-N long-only** o **long/short muy simple**. Qlib trae ejemplos de LightGBM y un workflow completo de benchmark/backtest que encaja muy bien con esa primera versión. qlib.readthedocs.io +2

Arquitectura recomendada

1) Research layer

Aquí vive Qlib.

Responsabilidades:

- dataset
- features
- labels
- entrenamiento
- validación temporal
- backtest
- análisis de señales

2) Signal layer

Una capa tuya, ligera.

Responsabilidades:

- leer predicciones del modelo
- rankear activos
- elegir top N

- convertir score en pesos

3) Execution layer

También tuya, fuera de Qlib.

Responsabilidades:

- leer pesos objetivo
- comparar contra cartera actual
- generar órdenes
- enviar a paper broker / API
- guardar fills, errores y estado

4) Monitoring layer

Responsabilidades:

- logs
- alertas
- equity curve
- drawdown
- posiciones
- divergencias entre señal y ejecución

Estructura de carpetas

Te propongo algo así:

```
auto_trading_system/
|
|— README.md
|— requirements.txt
|— .env
|
|— conf/
|   |— qlib_init.yaml
|   |— workflow_lightgbm.yaml
|   |— strategy.yaml
|   |— risk.yaml
|
|— data/
|   |— raw/
|   |— qlib_data/
|   |— signals/
|   |— orders/
|   |— reports/
|
|— research/
```



```
|
|   ├── run_workflow.py
|   ├── rolling_train.py
|   ├── export_signals.py
|   └── feature_defs.py
|
|── portfolio/
|   ├── ranking.py
|   ├── sizing.py
|   ├── constraints.py
|   └── rebalance.py
|
|── execution/
|   ├── broker_interface.py
|   ├── paper_broker.py
|   ├── order_manager.py
|   └── executor.py
|
|── infra/
|   ├── db.py
|   ├── scheduler.py
|   ├── logging_conf.py
|   └── alerts.py
|
|── tests/
|   ├── test_ranking.py
|   ├── test_sizing.py
|   └── test_executor.py
|
|── notebooks/
|   └── research_sandbox.ipynb
```

Qué hace cada módulo

research/run_workflow.py

Lanza el workflow Qlib benchmark o tu config propia.

Usa:

- inicialización Qlib
- carga de config
- entrenamiento
- backtest
- persistencia de artefactos

Qlib documenta dos caminos muy claros para empezar: ejecutar un benchmark con `qrun` y también trabajar "by code" con ejemplos en `examples` . [GitHub](#) +1

research/export_signals.py

Convierte la salida del modelo en un archivo simple, por ejemplo:

```
date,instrument,score
2026-03-24,AAPL,0.0231
2026-03-24,MSFT,0.0194
...
```

Este archivo es el contrato entre research y ejecución.

`portfolio/ranking.py`

Ordena activos por score y selecciona el universo invertible del día.

Ejemplo:

- eliminar activos sin liquidez
- eliminar penny stocks
- quedarte con top 10

`portfolio/sizing.py`

Transforma ranking en pesos.

Primera versión:

- equal weight top N
- cap por posición: 10%
- cash residual si hay menos nombres válidos

`portfolio/constraints.py`

Aplica reglas de riesgo:

- peso máximo por activo
- máximo turnover
- filtro de volumen medio
- mínimo precio
- stop de portfolio
- máxima exposición neta

`execution/paper_broker.py`

Simula broker.

Funciones:

- posiciones actuales

- efectivo
- órdenes abiertas
- fills simulados
- comisiones
- slippage

`execution/executor.py`

Lee pesos objetivo y genera órdenes:

- objetivo actual vs cartera actual
- diferencia → órdenes
- envío al paper broker
- registro en base de datos

Flujo operativo diario

Un ciclo sencillo sería:

Después del cierre

1. actualizar datos
2. generar features
3. inferir scores del modelo
4. exportar señales
5. construir pesos objetivo

Antes de la apertura siguiente

6. leer cartera actual
7. calcular órdenes necesarias
8. aplicar filtros de riesgo
9. paper execute
10. guardar estado y métricas

Estrategia inicial recomendada

Universo

Empieza con uno pequeño y limpio:

- 20 a 100 acciones líquidas
- o ETFs líquidos

Frecuencia

- diaria

Label

Ejemplo práctico:

- retorno a 5 días
- o ranking cross-sectional de retorno futuro

Modelo

- LightGBM

Qlib usa LightGBM en su quick start y benchmarks como punto de entrada estándar para un workflow completo. qlib.readthedocs.io +2

Construcción de cartera

Versión 1:

- top 10 scores
- equal weight
- rebalance diario
- comisión fija + slippage simple

Reglas de seguridad

- no operar si faltan datos
- no operar si el turnover supera cierto umbral
- no operar activos por debajo de cierto volumen
- no aumentar una posición si ya está en límite

Roadmap técnico por fases

Fase 1 — levantar Qlib y reproducir un benchmark

Objetivo: demostrar que tu entorno funciona.

Tareas:

- instalar `pyqlib`
- preparar datos Qlib
- lanzar benchmark oficial LightGBM
- revisar métricas y outputs

Qlib mantiene quick start y ejemplos precisamente para eso, incluyendo el comando `grun benchmarks/LightGBM/workflow_config_lightgbm.yaml` . [GitHub](#) +1

Entregable: un backtest reproducible corriendo en tu máquina.

Fase 2 — entender y clonar el workflow

Objetivo: dejar de depender del benchmark "caja negra".

Tareas:

- copiar config base
- revisar dataset, segments y label
- identificar dónde cambias features
- ejecutar con cambios mínimos

Entregable: tu propia `workflow_lightgbm.yaml` .

Fase 3 — exportar señales

Objetivo: desacoplar research de ejecución.

Tareas:

- leer predicciones del modelo
- guardarlas en CSV/Parquet/SQLite
- crear un formato estable de señal diaria

Formato sugerido:

```
date
instrument
score
rank
target_weight
model_version
```



Entregable: un fichero diario de señal listo para consumir.

Fase 4 — capa de portfolio

Objetivo: convertir scores en decisiones operables.

Tareas:

- top N
- normalización de pesos
- caps por posición
- filtro de liquidez
- control de turnover

Entregable: `target_positions.parquet`

Fase 5 — paper broker

Objetivo: simular ejecución sin dinero real.

Tareas:

- modelar efectivo inicial
- modelar comisiones
- modelar slippage
- registrar fills
- actualizar cartera

Entregable: un paper trading loop completo.

Fase 6 — scheduler + monitorización

Objetivo: automatización.

Tareas:

- cron / scheduler
- logging
- alertas Telegram/email
- reporte diario de PnL, exposición y órdenes

Entregable: sistema que corre solo cada día.

MVP mínimo viable en código

Tu primera versión puede quedar así:

`research/run_workflow.py`

- inicializa Qlib
- ejecuta entrenamiento / inferencia
- guarda scores

`portfolio/rebalance.py`

- lee scores del día
- construye target weights
- escribe órdenes objetivo

execution/executor.py

- lee target weights
- consulta cartera actual
- genera órdenes
- paper execute
- guarda resultado

Contrato de datos entre módulos

Haz esto simple y explícito.

signals.parquet

date, instrument, score, model_version



target_positions.parquet

date, instrument, target_weight



executed_orders.parquet

timestamp, instrument, side, qty, price, fee, status



portfolio_snapshot.parquet

date, instrument, qty, market_value, weight



Métricas que debes vigilar desde el día 1

No te quedes solo con retorno.

Mira:

- CAGR
- Sharpe
- max drawdown
- turnover
- hit ratio
- IC / Rank IC si trabajas cross-sectional
- coste neto después de comisiones
- estabilidad por periodo

Qlib incluye análisis y reporting del workflow/backtest, así que úsalo para la parte de research y añade tus métricas operativas en la capa de ejecución. [GitHub](#) +1

Riesgos típicos que debes evitar

1. Mezclar research y live execution

Error clásico. Mejor mantener dos capas separadas.

2. Sobreajuste

Haz splits estrictamente temporales y rolling / walk-forward. Qlib tiene documentación y ejemplos de rolling process y workflows reproducibles; aprovéchalos para no validar "mal" por accidente. [GitHub](#) +1

3. Costes irreales

Añade comisión y slippage desde la primera semana.

4. Universo sucio

No metas cientos de activos desde el principio. Empieza pequeño.

5. Exceso de complejidad

Nada de RL, intradía ni ejecución sofisticada al principio, aunque Qlib tenga componentes para ello. El propio repositorio muestra que esos módulos existen, pero eso no significa que sean el mejor punto de arranque para tu MVP. [GitHub](#) +1

Stack recomendado

- **Qlib** para research, dataset, training, backtest [GitHub](#) +1
- **LightGBM** como primer modelo, apoyado por los benchmarks oficiales de Qlib
qlib.readthedocs.io +1
- **pandas / numpy** para glue code
- **SQLite o Postgres** para órdenes y snapshots
- **cron / APScheduler** para automatización
- **Telegram/email** para alertas
- **Parquet** para señales y posiciones

Plan de 4 semanas

Semana 1

- instalar y correr benchmark LightGBM
- entender config
- guardar primer reporte

Semana 2

- clonar config
- definir universo y label
- exportar señales diarias

Semana 3

- construir portfolio layer
- paper broker
- ejecutar rebalance automático

Semana 4

- scheduler
- logging
- métricas
- validación walk-forward
- reporte diario

Decisión de diseño que te recomiendo mantener

Haz que **Qlib** nunca mande órdenes directamente.

Haz que Qlib produzca solo:

- scores
- rankings
- target weights

Y que otro proceso haga:

- traducción a órdenes
- control de riesgo final
- ejecución
- reconciliación

Eso te dará un sistema mucho más mantenible.

Siguiente paso concreto

Tu primer hito debería ser este:

"Correr benchmark LightGBM de Qlib y exportar una tabla diaria con score por activo."

A partir de ahí, el resto ya es ingeniería de sistema.

En el siguiente mensaje te puedo dejar un **esqueleto real del proyecto** con:

- árbol de carpetas final
- requirements.txt
- workflow_lightgbm.yaml
- run_workflow.py
- export_signals.py
- paper_broker.py
- executor.py

para que arranques directamente en local.

     ...  Fuentes

adelante

Pensó durante 6s >

Perfecto. Te dejo un **esqueleto funcional de MVP** para arrancar en local con Qlib, pensado para este flujo:

Qlib entrena + genera scores → tu capa convierte scores en pesos → tu paper broker simula órdenes → tu executor corre el rebalance diario.

Esto encaja con el workflow oficial de Qlib, que usa `qrun` con configs tipo LightGBM para dataset, entrenamiento, backtest y evaluación. El repositorio oficial sigue mostrando ese enfoque como quick start, y la release más reciente visible del proyecto es la **v0.9.4** . qlib.readthedocs.io +2

Estructura final

```
auto_trading_system/
├── requirements.txt
├── .env.example
├── conf/
│   ├── workflow_lightgbm.yaml
│   └── strategy.yaml
├── data/
│   ├── signals/
│   ├── orders/
│   ├── state/
│   └── reports/
├── research/
└── run_workflow.py
```



```

├── export_signals.py
├── portfolio/
│   ├── ranking.py
│   └── rebalance.py
├── execution/
│   ├── paper_broker.py
│   └── executor.py
├── infra/
│   └── paths.py

```

requirements.txt

Qlib se instala como `pyqlib` ; la documentación oficial también recomienda instalar dependencias como `lightgbm` y usar entorno aislado. [PyPI](#) +1

```

pyqlib
lightgbm
pandas
numpy
pyyaml
pyarrow
loguru

```



conf/workflow_lightgbm.yaml

Este archivo está inspirado en el patrón oficial de benchmark LightGBM de Qlib con `qrun` , pero simplificado para un MVP. Qlib documenta precisamente configs de este tipo para lanzar el workflow automático. [qlib.readthedocs.io](#) +2

```

qlib_init:
  provider_uri: "~/.qlib/qlib_data/us_data"
  region: us

market: &market sp500
benchmark: &benchmark ^GSPC

data_handler_config: &data_handler_config
  start_time: 2018-01-01
  end_time: 2024-12-31
  fit_start_time: 2018-01-01
  fit_end_time: 2022-12-31
  instruments: *market

task:
  model:
    class: LGBModel
    module_path: qlib.contrib.model.gbd

```

```
kwargs:
  loss: mse
  colsample_bytree: 0.8
  learning_rate: 0.05
  subsample: 0.8
  lambda_l1: 1.0
  lambda_l2: 1.0
  max_depth: 8
  num_leaves: 64
  num_threads: 8

dataset:
  class: DatasetH
  module_path: qlib.data.dataset
  kwargs:
    handler:
      class: Alpha158
      module_path: qlib.contrib.data.handler
      kwargs: *data_handler_config
    segments:
      train: [2018-01-01, 2022-12-31]
      valid: [2023-01-01, 2023-12-31]
      test: [2024-01-01, 2024-12-31]

port_analysis_config:
  strategy:
    class: TopkDropoutStrategy
    module_path: qlib.contrib.strategy
    kwargs:
      signal: <PRED>
      topk: 10
      n_drop: 2

backtest:
  start_time: 2024-01-01
  end_time: 2024-12-31
  account: 100000
  benchmark: *benchmark
  exchange_kwargs:
    limit_threshold: 0.095
    deal_price: close
    open_cost: 0.0005
    close_cost: 0.0015
    min_cost: 5

experiment_name: qlib_mvp_lgbm
```

conf/strategy.yaml

```

top_n: 10
min_score: null
weight_mode: equal
max_weight: 0.10
min_price: 5.0
min_avg_dollar_volume: 1000000
max_turnover: 0.30
cash_buffer: 0.02

```

infra/paths.py

```

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
SIGNALS_DIR = DATA_DIR / "signals"
ORDERS_DIR = DATA_DIR / "orders"
STATE_DIR = DATA_DIR / "state"
REPORTS_DIR = DATA_DIR / "reports"

for folder in [SIGNALS_DIR, ORDERS_DIR, STATE_DIR, REPORTS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

```

research/run_workflow.py

Qlib documenta `qrun` como la forma directa de ejecutar el workflow completo. Aquí lo lanzamos desde Python para que puedas integrarlo mejor con tu proyecto.

qlib.readthedocs.io +1

```

from pathlib import Path
import subprocess
import sys

def main() -> None:
    base_dir = Path(__file__).resolve().parent.parent
    config_path = base_dir / "conf" / "workflow_lightgbm.yaml"

    cmd = ["qrun", str(config_path)]
    print("Running:", " ".join(cmd))
    result = subprocess.run(cmd, cwd=base_dir, check=False)

    if result.returncode != 0:
        raise SystemExit(f"qrun failed with exit code {result.returncode}")

```



```
print("Workflow finished successfully.")
```

```
if __name__ == "__main__":
    main()
```

research/export_signals.py

Este script asume que vas a generar o leer predicciones diarias y exportarlas a un formato estable. En un proyecto real, el origen puede ser el recorder de Qlib o un flujo "by code"; Qlib soporta ambos enfoques. qlib.readthedocs.io +1

```
from __future__ import annotations

from datetime import datetime
from pathlib import Path
import pandas as pd

from infra.paths import SIGNALS_DIR

def export_mock_signals(trade_date: str) -> Path:
    # Sustituye esto por la lectura real desde el recorder o predicciones
    rows = [
        {"date": trade_date, "instrument": "AAPL", "score": 0.021},
        {"date": trade_date, "instrument": "MSFT", "score": 0.019},
        {"date": trade_date, "instrument": "NVDA", "score": 0.018},
        {"date": trade_date, "instrument": "AMZN", "score": 0.017},
        {"date": trade_date, "instrument": "META", "score": 0.016},
        {"date": trade_date, "instrument": "GOOGL", "score": 0.015},
        {"date": trade_date, "instrument": "AVGO", "score": 0.014},
        {"date": trade_date, "instrument": "TSLA", "score": 0.013},
        {"date": trade_date, "instrument": "AMD", "score": 0.012},
        {"date": trade_date, "instrument": "NFLX", "score": 0.011},
        {"date": trade_date, "instrument": "ADBE", "score": 0.010},
    ]
    df = pd.DataFrame(rows).sort_values("score", ascending=False).reset_index()
    df["rank"] = df.index + 1
    df["model_version"] = "lgbm_v1"

    out_path = SIGNALS_DIR / f"signals_{trade_date}.parquet"
    df.to_parquet(out_path, index=False)
    return out_path

if __name__ == "__main__":
    today = datetime.utcnow().strftime("%Y-%m-%d")
```

```
path = export_mock_signals(today)
print(f"Signals exported to {path}")
```

portfolio/ranking.py

```
from __future__ import annotations

import pandas as pd

def select_top_n(
    signals: pd.DataFrame,
    top_n: int = 10,
    min_score: float | None = None,
) -> pd.DataFrame:
    df = signals.copy()

    if min_score is not None:
        df = df[df["score"] >= min_score].copy()

    df = df.sort_values("score", ascending=False).head(top_n).copy()
    df["rank"] = range(1, len(df) + 1)
    return df.reset_index(drop=True)
```

portfolio/rebalance.py

```
from __future__ import annotations

from pathlib import Path
import yaml
import pandas as pd

from infra.paths import ORDERS_DIR, SIGNALS_DIR
from portfolio.ranking import select_top_n

def load_strategy_config(config_path: str | Path) -> dict:
    with open(config_path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)

def build_target_positions(signal_path: str | Path, config_path: str | Path):
    cfg = load_strategy_config(config_path)
    signals = pd.read_parquet(signal_path)
```

```

selected = select_top_n(
    signals,
    top_n=cfg["top_n"],
    min_score=cfg.get("min_score"),
)

cash_buffer = float(cfg.get("cash_buffer", 0.02))
investable_weight = max(0.0, 1.0 - cash_buffer)

if len(selected) == 0:
    raise ValueError("No valid assets selected after filtering.")

weight = investable_weight / len(selected)
max_weight = float(cfg.get("max_weight", 1.0))
weight = min(weight, max_weight)

selected["target_weight"] = weight

remainder = investable_weight - selected["target_weight"].sum()
if remainder > 0:
    selected.loc[selected.index[0], "target_weight"] += remainder

trade_date = str(selected["date"].iloc[0])
out_path = ORDERS_DIR / f"target_positions_{trade_date}.parquet"
selected[["date", "instrument", "score", "rank", "target_weight"]].to_
    out_path, index=False
)
return out_path

if __name__ == "__main__":
    import sys

    signal_path = sys.argv[1]
    config_path = sys.argv[2]
    out = build_target_positions(signal_path, config_path)
    print(f"Target positions saved to {out}")

```

execution/paper_broker.py

```

from __future__ import annotations

from dataclasses import dataclass, asdict
from pathlib import Path
import json
from typing import Dict, List

```

```

@dataclass
class Fill:
    instrument: str
    side: str
    qty: int
    price: float
    fee: float
    status: str = "filled"

class PaperBroker:
    def __init__(
        self,
        initial_cash: float = 100000.0,
        fee_rate: float = 0.001,
        state_path: str | Path = "data/state/paper_broker_state.json",
    ) -> None:
        self.state_path = Path(state_path)
        self.initial_cash = initial_cash
        self.fee_rate = fee_rate
        self.state_path.parent.mkdir(parents=True, exist_ok=True)

        if self.state_path.exists():
            self._load()
        else:
            self.cash = initial_cash
            self.positions: Dict[str, int] = {}
            self._save()

    def _load(self) -> None:
        data = json.loads(self.state_path.read_text(encoding="utf-8"))
        self.cash = float(data["cash"])
        self.positions = {k: int(v) for k, v in data["positions"].items()}

    def _save(self) -> None:
        payload = {"cash": self.cash, "positions": self.positions}
        self.state_path.write_text(json.dumps(payload, indent=2), encoding="utf-8")

    def get_positions(self) -> Dict[str, int]:
        return dict(self.positions)

    def get_cash(self) -> float:
        return self.cash

    def place_market_order(self, instrument: str, side: str, qty: int, price: float) -> None:
        if qty <= 0:
            raise ValueError("qty must be positive")

        gross = qty * price
        fee = gross * self.fee_rate

```

```

if side == "buy":
    total_cost = gross + fee
    if total_cost > self.cash:
        return Fill(instrument, side, qty, price, fee, status="rej")
    self.cash -= total_cost
    self.positions[instrument] = self.positions.get(instrument, 0)

elif side == "sell":
    current_qty = self.positions.get(instrument, 0)
    if qty > current_qty:
        return Fill(instrument, side, qty, price, fee, status="rej")
    self.cash += gross - fee
    new_qty = current_qty - qty
    if new_qty == 0:
        self.positions.pop(instrument, None)
    else:
        self.positions[instrument] = new_qty
else:
    raise ValueError("side must be 'buy' or 'sell'")

self._save()
return Fill(instrument, side, qty, price, fee)

```

execution/executor.py

```

from __future__ import annotations

from dataclasses import asdict
from pathlib import Path
import pandas as pd

from execution.paper_broker import PaperBroker
from infra.paths import ORDERS_DIR, REPORTS_DIR, STATE_DIR

def generate_orders_from_targets(
    targets: pd.DataFrame,
    prices: dict[str, float],
    broker: PaperBroker,
) -> list[dict]:
    cash = broker.get_cash()
    current_positions = broker.get_positions()

    portfolio_value = cash
    for instrument, qty in current_positions.items():
        portfolio_value += qty * prices[instrument]

```

```

orders: list[dict] = []

target_qty = {}
for _, row in targets.iterrows():
    instrument = row["instrument"]
    weight = float(row["target_weight"])
    price = float(prices[instrument])
    dollars = portfolio_value * weight
    qty = int(dollars // price)
    target_qty[instrument] = qty

all_instruments = set(current_positions) | set(target_qty)

for instrument in sorted(all_instruments):
    current = current_positions.get(instrument, 0)
    target = target_qty.get(instrument, 0)
    delta = target - current

    if delta > 0:
        orders.append(
            {"instrument": instrument, "side": "buy", "qty": delta, "p
        )
    elif delta < 0:
        orders.append(
            {"instrument": instrument, "side": "sell", "qty": abs(delta
        )

return orders

def execute_target_positions(
    target_path: str | Path,
    prices: dict[str, float],
) -> Path:
    targets = pd.read_parquet(target_path)
    broker = PaperBroker(state_path=STATE_DIR / "paper_broker_state.json")

    orders = generate_orders_from_targets(targets, prices, broker)

    fills = []
    for order in orders:
        fill = broker.place_market_order(
            instrument=order["instrument"],
            side=order["side"],
            qty=int(order["qty"]),
            price=float(order["price"]),
        )
        fills.append(asdict(fill))

    out = pd.DataFrame(fills)
    trade_date = str(targets["date"].iloc[0])

```

```

out_path = REPORTS_DIR / f"fills_{trade_date}.parquet"
out.to_parquet(out_path, index=False)
return out_path

if __name__ == "__main__":
    import sys
    import json

    target_path = sys.argv[1]
    prices = json.loads(sys.argv[2]) # {"AAPL":190,"MSFT":420,...}
    result = execute_target_positions(target_path, prices)
    print(f"Fills saved to {result}")

```

Flujo de ejecución diario

```

python research/run_workflow.py
python research/export_signals.py
python portfolio/rebalance.py data/signals/signals_2026-03-24.parquet conf
python execution/executor.py data/orders/target_positions_2026-03-24.parqu

```

Qué te falta para convertir esto en algo serio

Lo mínimo que añadiría a continuación:

1. **Lectura real de predicciones desde Qlib recorder**, en vez de `export_mock_signals`.
2. **Precios reales del día** para calcular órdenes.
3. **Control de turnover** y límite de peso por activo.
4. **Filtro de liquidez** antes de construir pesos.
5. **Scheduler** con cron o APScheduler.
6. **Walk-forward retraining**, en vez de un único entrenamiento fijo.

Qlib está diseñado precisamente para research reproducible, workflows por config y evaluación/backtest; por eso te conviene mantener la ejecución real fuera de Qlib y dejarlo como motor de señal. [GitHub](#) +2

Orden de trabajo recomendado

Primero deja funcionando esto:

- `run_workflow.py`
- `export_signals.py`
- `rebalance.py`

- `executor.py`

Y cuando eso corra de punta a punta, haces la siguiente iteración:

- reemplazar mock signals por predicciones reales
- añadir base de datos SQLite
- generar informe diario de PnL y posiciones
- conectar paper trading con broker demo

El siguiente paso útil es que te monte una **versión 2 del esqueleto**, ya con:

- Makefile
- SQLite
- APScheduler
- logs con `loguru`
- y una implementación de `export_signals.py` conectada de verdad al recorder de Qlib.

     ...  Fuentes

adelante

Pensó durante 10s >

Perfecto. Vamos con una **v2 más seria**: misma arquitectura, pero añadiendo estas piezas:

- **SQLite** para estado, órdenes y fills
- **APScheduler** para automatización diaria
- **loguru** para logs
- **lectura real del recorder de Qlib** en `export_signals.py`

Qlib sigue documentando un workflow por código con `R`, `SignalRecord` y `PortAnaRecord`, y el recorder permite cargar artefactos como `pred.pkl` y `trained_model`. La guía rápida actual de Qlib sigue mostrando ese enfoque, y la documentación actual visible está en la rama `0.9.8.dev11`. APScheduler mantiene guía oficial tanto para la rama estable `3.x` como para la rama nueva "master"; para un MVP simple y predecible, me quedaría con **3.x**. [Qlib Documentati...](#) +4

Estructura v2

```
auto_trading_system/
├── requirements.txt
```




```
|— .env.example
|— run_daily.py
|— conf/
|   |— workflow_lightgbm.yaml
|   |— strategy.yaml
|— data/
|   |— signals/
|   |— orders/
|   |— reports/
|   |— logs/
|   |— state/
|— research/
|   |— run_workflow.py
|   |— export_signals.py
|— portfolio/
|   |— ranking.py
|   |— rebalance.py
|— execution/
|   |— paper_broker.py
|   |— executor.py
|— infra/
|   |— paths.py
|   |— db.py
|   |— logging_conf.py
|   |— scheduler.py
```

requirements.txt

```
pyqlib
lightgbm
pandas
numpy
pyyaml
pyarrow
loguru
apscheduler==3.11.0
```



infra/paths.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
```

```
SIGNALS_DIR = DATA_DIR / "signals"
ORDERS_DIR = DATA_DIR / "orders"
REPORTS_DIR = DATA_DIR / "reports"
LOGS_DIR = DATA_DIR / "logs"
STATE_DIR = DATA_DIR / "state"

for folder in [SIGNALS_DIR, ORDERS_DIR, REPORTS_DIR, LOGS_DIR, STATE_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

DB_PATH = STATE_DIR / "trading.db"
```

infra/logging_conf.py

```
from loguru import logger
from infra.paths import LOGS_DIR

def setup_logging() -> None:
    logger.remove()
    logger.add(
        LOGS_DIR / "system.log",
        rotation="10 MB",
        retention=10,
        level="INFO",
        enqueue=True,
        backtrace=True,
        diagnose=False,
    )
    logger.add(
        lambda msg: print(msg, end=""),
        level="INFO",
    )
```

infra/db.py

```
import sqlite3
from contextlib import contextmanager
from infra.paths import DB_PATH
```

```
SCHEMA = """"
CREATE TABLE IF NOT EXISTS signals (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    score REAL NOT NULL,
    rank INTEGER,
    model_version TEXT,
    PRIMARY KEY (date, instrument, model_version)
);

CREATE TABLE IF NOT EXISTS target_positions (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    target_weight REAL NOT NULL,
    score REAL,
    rank INTEGER,
    PRIMARY KEY (date, instrument)
);

CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    side TEXT NOT NULL,
    qty INTEGER NOT NULL,
    price REAL NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS fills (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    side TEXT NOT NULL,
    qty INTEGER NOT NULL,
    price REAL NOT NULL,
    fee REAL NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS broker_state (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
""""
```

```
@contextmanager
def get_conn():
```

```

conn = sqlite3.connect(DB_PATH)
conn.row_factory = sqlite3.Row
try:
    yield conn
    conn.commit()
finally:
    conn.close()

def init_db() -> None:
    with get_conn() as conn:
        conn.executescript(SCHEMA)

```

research/run_workflow.py

Aquí seguimos usando `qrun`, porque Qlib lo mantiene como camino estándar de quick start para workflows por configuración. [Qlib Documentati...](#) +1

```

from pathlib import Path
import subprocess

from loguru import logger

def main() -> None:
    base_dir = Path(__file__).resolve().parent.parent
    config_path = base_dir / "conf" / "workflow_lightgbm.yaml"

    cmd = ["qrun", str(config_path)]
    logger.info(f"Running workflow: {' '.join(cmd)}")

    result = subprocess.run(cmd, cwd=base_dir, check=False)
    if result.returncode != 0:
        raise SystemExit(f"qrun failed with exit code {result.returncode}")

    logger.info("Workflow completed successfully.")

if __name__ == "__main__":
    main()

```

research/export_signals.py

Esta versión ya no usa mock. Usa el recorder de Qlib para cargar `pred.pkl`, que es precisamente el patrón mostrado en los ejemplos oficiales de workflow por código.

GitHub +1

```
from __future__ import annotations

from pathlib import Path
import pandas as pd
import qlib
from qlib.workflow import R
from loguru import logger

from infra.paths import SIGNALS_DIR
from infra.db import get_conn

def export_latest_predictions(
    experiment_name: str,
    recorder_id: str | None = None,
    provider_uri: str = "~/.qlib/qlib_data/us_data",
    region: str = "us",
    model_version: str = "lgbm_v2",
) -> Path:
    qlib.init(provider_uri=provider_uri, region=region)

    if recorder_id:
        recorder = R.get_recorder(recorder_id=recorder_id, experiment_name=experiment_name)
    else:
        recorder = R.get_recorder(experiment_name=experiment_name)

    pred_df = recorder.load_object("pred.pkl")

    if isinstance(pred_df, pd.Series):
        pred_df = pred_df.to_frame("score")
    elif "score" not in pred_df.columns:
        pred_df.columns = ["score"]

    pred_df = pred_df.reset_index()

    # Esperamos columnas tipo: datetime, instrument, score
    if "datetime" in pred_df.columns:
        pred_df["date"] = pd.to_datetime(pred_df["datetime"]).dt.strftime("%Y-%m-%d")
    elif "date" in pred_df.columns:
        pred_df["date"] = pd.to_datetime(pred_df["date"]).dt.strftime("%Y-%m-%d")
    else:
        raise ValueError("Prediction dataframe has no datetime/date column")

    latest_date = pred_df["date"].max()
    latest = pred_df[pred_df["date"] == latest_date].copy()
```

```

if "instrument" not in latest.columns:
    raise ValueError("Prediction dataframe has no instrument column")

latest = latest.sort_values("score", ascending=False).reset_index(drop=True)
latest["rank"] = latest.index + 1
latest["model_version"] = model_version

out = latest[["date", "instrument", "score", "rank", "model_version"]]
out_path = SIGNALS_DIR / f"signals_{latest_date}.parquet"
out.to_parquet(out_path, index=False)

with get_conn() as conn:
    conn.executemany(
        """
        INSERT OR REPLACE INTO signals(date, instrument, score, rank,
        VALUES (?, ?, ?, ?, ?)
        """,
        out.itertuples(index=False, name=None),
    )

logger.info(f"Exported {len(out)} signals for {latest_date} to {out_path}")
return out_path

if __name__ == "__main__":
    export_latest_predictions(experiment_name="backtest_analysis")

```

portfolio/ranking.py

```

from __future__ import annotations
import pandas as pd

def select_top_n(
    signals: pd.DataFrame,
    top_n: int,
    min_score: float | None = None,
) -> pd.DataFrame:
    df = signals.copy()

    if min_score is not None:
        df = df[df["score"] >= min_score].copy()

    df = df.sort_values("score", ascending=False).head(top_n).copy()

```

```
df["rank"] = range(1, len(df) + 1)
return df.reset_index(drop=True)
```

portfolio/rebalance.py

```
from __future__ import annotations

from pathlib import Path
import yaml
import pandas as pd
from loguru import logger

from infra.paths import ORDERS_DIR
from infra.db import get_conn
from portfolio.ranking import select_top_n


def load_config(config_path: str | Path) -> dict:
    with open(config_path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)


def build_target_positions(signal_path: str | Path, config_path: str | Path):
    cfg = load_config(config_path)
    signals = pd.read_parquet(signal_path)

    selected = select_top_n(
        signals,
        top_n=int(cfg["top_n"]),
        min_score=cfg.get("min_score"),
    )

    if selected.empty:
        raise ValueError("No assets selected")

    cash_buffer = float(cfg.get("cash_buffer", 0.02))
    investable = 1.0 - cash_buffer

    weight_mode = cfg.get("weight_mode", "equal")
    max_weight = float(cfg.get("max_weight", 1.0))

    if weight_mode != "equal":
        raise NotImplementedError("Only equal weight is implemented in v2")

    base_weight = min(investable / len(selected), max_weight)
    selected["target_weight"] = base_weight
```

```

remainder = investable - selected["target_weight"].sum()
if remainder > 0:
    selected.loc[selected.index[0], "target_weight"] += remainder

trade_date = selected["date"].iloc[0]
out = selected[["date", "instrument", "target_weight", "score", "rank"]
out_path = ORDERS_DIR / f"target_positions_{trade_date}.parquet"
out.to_parquet(out_path, index=False)

with get_conn() as conn:
    conn.executemany(
        """
        INSERT OR REPLACE INTO target_positions(date, instrument, targ
        VALUES (?, ?, ?, ?, ?)
        """,
        out.itertuples(index=False, name=None),
    )

logger.info(f"Built {len(out)} target positions for {trade_date}")
return out_path

```

execution/paper_broker.py

```

from __future__ import annotations

import json
from dataclasses import dataclass
from loguru import logger

from infra.db import get_conn

@dataclass
class Fill:
    instrument: str
    side: str
    qty: int
    price: float
    fee: float
    status: str

class PaperBroker:
    def __init__(self, initial_cash: float = 100000.0, fee_rate: float = 0
    self.initial_cash = initial_cash

```



```

self.fee_rate = fee_rate
self._bootstrap()

def _bootstrap(self) -> None:
    with get_conn() as conn:
        rows = conn.execute("SELECT key, value FROM broker_state").fetchall()
        if not rows:
            conn.execute(
                "INSERT OR REPLACE INTO broker_state(key, value) VALUES ("
                "cash", str(self.initial_cash)),
            )
            conn.execute(
                "INSERT OR REPLACE INTO broker_state(key, value) VALUES ("
                "positions", json.dumps({})),
            )

def get_cash(self) -> float:
    with get_conn() as conn:
        row = conn.execute("SELECT value FROM broker_state WHERE key='cash'").fetchall()
        return float(row[0])

def set_cash(self, cash: float) -> None:
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO broker_state(key, value) VALUES ("
            "cash", str(cash)),
        )

def get_positions(self) -> dict[str, int]:
    with get_conn() as conn:
        row = conn.execute("SELECT value FROM broker_state WHERE key='positions'").fetchall()
        return json.loads(row[0])

def set_positions(self, positions: dict[str, int]) -> None:
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO broker_state(key, value) VALUES ("
            "positions", json.dumps(positions)),
        )

def place_market_order(self, instrument: str, side: str, qty: int, price: float) -> None:
    if qty <= 0:
        raise ValueError("qty must be positive")

    cash = self.get_cash()
    positions = self.get_positions()
    gross = qty * price
    fee = gross * self.fee_rate

    if side == "buy":
        total = gross + fee
        if total > cash:

```

```

        return Fill(instrument, side, qty, price, fee, "rejected_n
cash -= total
positions[instrument] = int(positions.get(instrument, 0)) + qt

elif side == "sell":
    current = int(positions.get(instrument, 0))
    if qty > current:
        return Fill(instrument, side, qty, price, fee, "rejected_n
cash += gross - fee
remaining = current - qty
if remaining == 0:
    positions.pop(instrument, None)
else:
    positions[instrument] = remaining
else:
    raise ValueError("side must be buy/sell")

self.set_cash(cash)
self.set_positions(positions)
logger.info(f"Filled {side} {qty} {instrument} @ {price:.2f}")
return Fill(instrument, side, qty, price, fee, "filled")

```

execution/executor.py

```

from __future__ import annotations

from pathlib import Path
import json
import pandas as pd
from loguru import logger

from execution.paper_broker import PaperBroker
from infra.db import get_conn
from infra.paths import REPORTS_DIR

def generate_orders_from_targets(
    targets: pd.DataFrame,
    prices: dict[str, float],
    broker: PaperBroker,
) -> list[dict]:
    cash = broker.get_cash()
    positions = broker.get_positions()

    portfolio_value = cash

```

```

for instrument, qty in positions.items():
    if instrument in prices:
        portfolio_value += int(qty) * float(prices[instrument])

target_qty = {}
for _, row in targets.iterrows():
    instrument = row["instrument"]
    weight = float(row["target_weight"])
    price = float(prices[instrument])
    dollars = portfolio_value * weight
    target_qty[instrument] = int(dollars // price)

all_instruments = set(positions) | set(target_qty)
orders = []

for instrument in sorted(all_instruments):
    current = int(positions.get(instrument, 0))
    target = int(target_qty.get(instrument, 0))
    delta = target - current

    if delta > 0:
        orders.append(
            {"instrument": instrument, "side": "buy", "qty": delta, "p
        )
    elif delta < 0:
        orders.append(
            {"instrument": instrument, "side": "sell", "qty": abs(delta
        )

return orders

def execute_target_positions(target_path: str | Path, prices: dict[str, fl
broker = PaperBroker()
targets = pd.read_parquet(target_path)
trade_date = str(targets["date"].iloc[0])

orders = generate_orders_from_targets(targets, prices, broker)

fills = []
with get_conn() as conn:
    for order in orders:
        conn.execute(
            """
            INSERT INTO orders(date, instrument, side, qty, price, sta
            VALUES (?, ?, ?, ?, ?, ?)
            """,
            (
                trade_date,
                order["instrument"],
                order["side"],
                int(order["qty"]),

```

```

        float(order["price"]),
        "created",
    ),
)

fill = broker.place_market_order(
    instrument=order["instrument"],
    side=order["side"],
    qty=int(order["qty"]),
    price=float(order["price"]),
)

conn.execute(
    """
    INSERT INTO fills(date, instrument, side, qty, price, fee,
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """,
    (
        trade_date,
        fill.instrument,
        fill.side,
        fill.qty,
        fill.price,
        fill.fee,
        fill.status,
    ),
)

fills.append(
    {
        "date": trade_date,
        "instrument": fill.instrument,
        "side": fill.side,
        "qty": fill.qty,
        "price": fill.price,
        "fee": fill.fee,
        "status": fill.status,
    }
)

out = pd.DataFrame(fills)
out_path = REPORTS_DIR / f"fills_{trade_date}.parquet"
out.to_parquet(out_path, index=False)
logger.info(f"Saved fills to {out_path}")
return out_path

if __name__ == "__main__":
    import sys

    target_path = sys.argv[1]

```

```
prices = json.loads(sys.argv[2])
execute_target_positions(target_path, prices)
```

infra/scheduler.py

Para una automatización local simple, APScheduler 3.x sigue documentando BackgroundScheduler y la configuración de jobs/executors desde el scheduler.

APScheduler +1

```
from apscheduler.schedulers.background import BackgroundScheduler
from loguru import logger

def build_scheduler():
    scheduler = BackgroundScheduler(timezone="Europe/Madrid")
    return scheduler

def start_scheduler(job_func):
    scheduler = build_scheduler()
    scheduler.add_job(
        job_func,
        trigger="cron",
        hour=22,
        minute=15,
        id="daily_trading_job",
        replace_existing=True,
    )
    scheduler.start()
    logger.info("Scheduler started: daily job at 22:15 Europe/Madrid")
    return scheduler
```

run_daily.py

Este es el orquestador.

```
from infra.logging_conf import setup_logging
from infra.db import init_db
from research.run_workflow import main as run_workflow
from research.export_signals import export_latest_predictions
```

```
from portfolio.rebalance import build_target_positions
from execution.executor import execute_target_positions

def fetch_mock_prices():
    return {
        "AAPL": 190.0,
        "MSFT": 420.0,
        "NVDA": 880.0,
        "AMZN": 180.0,
        "META": 500.0,
        "GOOGL": 160.0,
        "AVGO": 1350.0,
        "TSLA": 175.0,
        "AMD": 165.0,
        "NFLX": 610.0,
        "ADBE": 540.0,
    }

def run_pipeline():
    run_workflow()
    signal_path = export_latest_predictions(experiment_name="backtest_anal
    target_path = build_target_positions(signal_path, "conf/strategy.yaml"
    prices = fetch_mock_prices()
    execute_target_positions(target_path, prices)

if __name__ == "__main__":
    setup_logging()
    init_db()
    run_pipeline()
```

conf/strategy.yaml

```
top_n: 10
min_score: null
weight_mode: equal
max_weight: 0.10
cash_buffer: 0.02
```

Cómo lanzar el sistema

```
pip install -r requirements.txt
python run_daily.py
```

Y si quieres dejarlo corriendo con scheduler:

```
from time import sleep
from infra.logging_conf import setup_logging
from infra.db import init_db
from infra.scheduler import start_scheduler
from run_daily import run_pipeline

setup_logging()
init_db()
scheduler = start_scheduler(run_pipeline)

while True:
    sleep(60)
```

Qué debes revisar en Qlib para que esto encaje

En tu workflow Qlib, asegúrate de que el experimento que luego lees con `export_latest_predictions()` realmente haya generado `pred.pkl`. En los ejemplos oficiales eso ocurre cuando usas `SignalRecord(...).generate()` dentro del experimento de backtest/análisis. El ejemplo oficial también muestra cargar luego `pred.pkl` desde el recorder. [GitHub](#) +1

Qué mejoraría en la v3

Lo siguiente que yo haría ya no es más "esqueleto", sino hacerlo usable:

- sustituir `fetch_mock_prices()` por precios reales
- guardar **equity diaria** en SQLite
- añadir **turnover cap**
- añadir **slippage configurable**
- reentrenamiento **walk-forward**
- alertas por Telegram o email
- reconciliación entre señal objetivo y cartera real

Lo importante de esta versión

Ya tienes separado:

- **Qlib** para entrenar y producir señales
- **portfolio layer** para convertir score en pesos
- **execution layer** para simular órdenes
- **scheduler + db + logs** para operar como sistema

Ese desacoplamiento encaja con cómo Qlib presenta su workflow modular: research reproducible, recorder de artefactos y backtesting/análisis, mientras tu capa externa se ocupa de la operativa diaria. [GitHub](#) +2

En el siguiente paso te puedo dejar una **v3 con precios reales via Yahoo/Stooq/Alpha Vantage y cálculo de equity/PnL diario**.

     ...  Fuentes

adelante

Pensó durante 8s >

Perfecto. Vamos con una **v3 usable**: precios reales, equity diaria y PnL.

La decisión de diseño que te recomiendo es esta:

- **por defecto** usa `yfinance` porque es muy cómodo y expone `download()` para históricos de varios tickers, pero recuerda que es un proyecto open source no afiliado a Yahoo y orientado a investigación/uso educativo [ranaroussi.github...](#) +2
- **si quieres algo más "formal"** añade un adaptador para **Alpha Vantage**, cuya API oficial ofrece `TIME_SERIES_DAILY` y `TIME_SERIES_DAILY_ADJUSTED` para acciones y requiere API key [alphavantage.co](#)
- Qlib sigue encajando como motor de research, recorder y generación de predicciones/artefactos del workflow [Qlib Documentati...](#) +2

Abajo te dejo una versión concreta con:

- `market_data.py`
- `prices.py`
- equity curve en SQLite
- snapshot diario de cartera
- cálculo de PnL diario
- soporte para `yfinance` y opcionalmente Alpha Vantage

Cambios de estructura

```
auto_trading_system/
├── requirements.txt
├── .env.example
├── run_daily.py
├── conf/
│   ├── workflow_lightgbm.yaml
│   └── strategy.yaml
├── data/
│   ├── signals/
│   ├── orders/
│   ├── reports/
│   ├── logs/
│   └── state/
├── research/
│   ├── run_workflow.py
│   └── export_signals.py
├── portfolio/
│   ├── ranking.py
│   └── rebalance.py
├── execution/
│   ├── paper_broker.py
│   ├── executor.py
│   └── pricing.py
├── market_data/
│   └── providers.py
└── infra/
    ├── paths.py
    ├── db.py
    ├── logging_conf.py
    └── scheduler.py
```



requirements.txt

yfinance expone una API pública con Ticker, Tickers y download() en su documentación; APScheduler 3.x sigue publicado y documentado como rama estable 3.x. ranaroussi.github... +2

```
pyqlib
lightgbm
pandas
numpy
pyyaml
pyarrow
loguru
apscheduler==3.11.0
yfinance
```



```
requests
python-dotenv
```

`.env.example`

```
MARKET_DATA_PROVIDER=yfinance
ALPHA_VANTAGE_API_KEY=tu_api_key
QLIB_PROVIDER_URI=~/.qlib/qlib_data/us_data
QLIB_REGION=us
INITIAL_CASH=100000
FEE_RATE=0.001
```



`infra/db.py`

Añadimos tablas para equity, snapshots y precios.

```
import sqlite3
from contextlib import contextmanager
from infra.paths import DB_PATH

SCHEMA = """
CREATE TABLE IF NOT EXISTS signals (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    score REAL NOT NULL,
    rank INTEGER,
    model_version TEXT,
    PRIMARY KEY (date, instrument, model_version)
);

CREATE TABLE IF NOT EXISTS target_positions (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    target_weight REAL NOT NULL,
    score REAL,
    rank INTEGER,
    PRIMARY KEY (date, instrument)
);

CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    side TEXT NOT NULL,
    qty INTEGER NOT NULL,
    price REAL NOT NULL,
    status TEXT NOT NULL,
```

```
        created_at TEXT DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE IF NOT EXISTS fills (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    side TEXT NOT NULL,
    qty INTEGER NOT NULL,
    price REAL NOT NULL,
    fee REAL NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS broker_state (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS daily_prices (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    close REAL NOT NULL,
    source TEXT NOT NULL,
    PRIMARY KEY (date, instrument, source)
);

CREATE TABLE IF NOT EXISTS portfolio_snapshots (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    qty INTEGER NOT NULL,
    close REAL NOT NULL,
    market_value REAL NOT NULL,
    weight REAL NOT NULL,
    PRIMARY KEY (date, instrument)
);

CREATE TABLE IF NOT EXISTS equity_curve (
    date TEXT PRIMARY KEY,
    cash REAL NOT NULL,
    gross_market_value REAL NOT NULL,
    net_liquidation_value REAL NOT NULL,
    daily_pnl REAL,
    daily_return REAL
);
"""

@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
```

```

conn.row_factory = sqlite3.Row
try:
    yield conn
    conn.commit()
finally:
    conn.close()

def init_db() -> None:
    with get_conn() as conn:
        conn.executescript(SCHEMA)

```

market_data/providers.py

Aquí tienes dos proveedores. `yfinance` va muy bien para empezar, pero no es una API oficial de Yahoo; Alpha Vantage sí documenta oficialmente sus endpoints de series diarias. [ranaroussi.github...](#) +1

```

from __future__ import annotations

import os
from dataclasses import dataclass
from typing import Iterable
import requests
import pandas as pd
import yfinance as yf

@dataclass
class PriceProviderConfig:
    provider: str = "yfinance"
    alpha_vantage_api_key: str | None = None

class BasePriceProvider:
    source_name: str = "base"

    def get_latest_closes(self, tickers: Iterable[str]) -> dict[str, float]:
        raise NotImplementedError

class YFinancePriceProvider(BasePriceProvider):
    source_name = "yfinance"

    def get_latest_closes(self, tickers: Iterable[str]) -> dict[str, float]:
        tickers = list(dict.fromkeys(tickers))
        if not tickers:
            return {}

```

```

df = yf.download(
    tickers=tickers,
    period="7d",
    interval="1d",
    auto_adjust=False,
    progress=False,
    group_by="ticker",
    threads=True,
)

prices: dict[str, float] = {}

if len(tickers) == 1:
    ticker = tickers[0]
    close_series = df["Close"].dropna()
    if close_series.empty:
        raise ValueError(f"No close data for {ticker}")
    prices[ticker] = float(close_series.iloc[-1])
    return prices

for ticker in tickers:
    close_series = df[ticker]["Close"].dropna()
    if close_series.empty:
        continue
    prices[ticker] = float(close_series.iloc[-1])

return prices

class AlphaVantagePriceProvider(BasePriceProvider):
    source_name = "alphavantage"

    def __init__(self, api_key: str) -> None:
        if not api_key:
            raise ValueError("Alpha Vantage API key is required")
        self.api_key = api_key

    def get_latest_closes(self, tickers: Iterable[str]) -> dict[str, float]:
        prices: dict[str, float] = {}

        for ticker in tickers:
            url = "https://www.alphavantage.co/query"
            params = {
                "function": "TIME_SERIES_DAILY_ADJUSTED",
                "symbol": ticker,
                "apikey": self.api_key,
                "outputsize": "compact",
                "datatype": "json",
            }
            resp = requests.get(url, params=params, timeout=30)
            resp.raise_for_status()
            payload = resp.json()

```

```

        series = payload.get("Time Series (Daily)")
        if not series:
            continue

        latest_date = max(series.keys())
        prices[ticker] = float(series[latest_date]["5. adjusted close"])

    return prices

def build_price_provider() -> BasePriceProvider:
    provider = os.getenv("MARKET_DATA_PROVIDER", "yfinance").lower()

    if provider == "yfinance":
        return YFinancePriceProvider()

    if provider == "alphavantage":
        api_key = os.getenv("ALPHA_VANTAGE_API_KEY")
        return AlphaVantagePriceProvider(api_key=api_key)

    raise ValueError(f"Unsupported provider: {provider}")

```

execution/pricing.py

Persistimos precios del día.

```

from __future__ import annotations

from loguru import logger
from infra.db import get_conn
from market_data.providers import build_price_provider

def fetch_and_store_latest_prices(trade_date: str, tickers: list[str]) -> None:
    provider = build_price_provider()
    prices = provider.get_latest_closes(tickers)

    if not prices:
        raise ValueError("No prices fetched")

    with get_conn() as conn:
        conn.executemany(
            """
            INSERT OR REPLACE INTO daily_prices(date, instrument, close, s
            VALUES (?, ?, ?, ?)
            """,
            [(trade_date, ticker, float(price), provider.source_name) for

```

```
)

logger.info(f"Stored {len(prices)} prices for {trade_date} from {provi
return prices
```

execution/paper_broker.py

La base sigue igual, pero añadimos métodos de valoración.

```
from __future__ import annotations

import json
from dataclasses import dataclass
from loguru import logger

from infra.db import get_conn

@dataclass
class Fill:
    instrument: str
    side: str
    qty: int
    price: float
    fee: float
    status: str

class PaperBroker:
    def __init__(self, initial_cash: float = 100000.0, fee_rate: float = 0):
        self.initial_cash = initial_cash
        self.fee_rate = fee_rate
        self._bootstrap()

    def _bootstrap(self) -> None:
        with get_conn() as conn:
            rows = conn.execute("SELECT key, value FROM broker_state").fetchall()
            if not rows:
                conn.execute(
                    "INSERT OR REPLACE INTO broker_state(key, value) VALUES ("
                    "cash", str(self.initial_cash)),
                )
                conn.execute(
                    "INSERT OR REPLACE INTO broker_state(key, value) VALUES ("
                    "positions", json.dumps({})),
                )

    def get_cash(self) -> float:
```

```

with get_conn() as conn:
    row = conn.execute("SELECT value FROM broker_state WHERE key='
    return float(row["value"])

def set_cash(self, cash: float) -> None:
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO broker_state(key, value) VALUES (?
            ("cash", str(cash)),
        )

def get_positions(self) -> dict[str, int]:
    with get_conn() as conn:
        row = conn.execute("SELECT value FROM broker_state WHERE key='
        return json.loads(row["value"])

def set_positions(self, positions: dict[str, int]) -> None:
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO broker_state(key, value) VALUES (?
            ("positions", json.dumps(positions)),
        )

def mark_to_market(self, prices: dict[str, float]) -> tuple[float, flo
    cash = self.get_cash()
    positions = self.get_positions()
    gross_market_value = 0.0

    for instrument, qty in positions.items():
        if instrument not in prices:
            continue
        gross_market_value += int(qty) * float(prices[instrument])

    net_liquidation_value = cash + gross_market_value
    return cash, gross_market_value, net_liquidation_value

def place_market_order(self, instrument: str, side: str, qty: int, pri
    if qty <= 0:
        raise ValueError("qty must be positive")

    cash = self.get_cash()
    positions = self.get_positions()
    gross = qty * price
    fee = gross * self.fee_rate

    if side == "buy":
        total = gross + fee
        if total > cash:
            return Fill(instrument, side, qty, price, fee, "rejected_n
        cash -= total
        positions[instrument] = int(positions.get(instrument, 0)) + qt

```



```

elif side == "sell":
    current = int(positions.get(instrument, 0))
    if qty > current:
        return Fill(instrument, side, qty, price, fee, "rejected_n
    cash += gross - fee
    remaining = current - qty
    if remaining == 0:
        positions.pop(instrument, None)
    else:
        positions[instrument] = remaining
else:
    raise ValueError("side must be buy/sell")

self.set_cash(cash)
self.set_positions(positions)
logger.info(f"Filled {side} {qty} {instrument} @ {price:.2f}")
return Fill(instrument, side, qty, price, fee, "filled")

```

execution/executor.py

Aquí ya calculamos órdenes, fills, snapshot y equity.

```

from __future__ import annotations

from pathlib import Path
import pandas as pd
from loguru import logger

from execution.paper_broker import PaperBroker
from infra.db import get_conn
from infra.paths import REPORTS_DIR

def generate_orders_from_targets(
    targets: pd.DataFrame,
    prices: dict[str, float],
    broker: PaperBroker,
) -> list[dict]:
    cash, gross_market_value, portfolio_value = broker.mark_to_market(pric
    positions = broker.get_positions()

    target_qty = {}
    for _, row in targets.iterrows():
        instrument = row["instrument"]
        if instrument not in prices:
            continue
        weight = float(row["target_weight"])
        price = float(prices[instrument])

```

```

dollars = portfolio_value * weight
target_qty[instrument] = int(dollars // price)

all_instruments = set(positions) | set(target_qty)
orders = []

for instrument in sorted(all_instruments):
    current = int(positions.get(instrument, 0))
    target = int(target_qty.get(instrument, 0))
    delta = target - current

    if instrument not in prices:
        continue

    if delta > 0:
        orders.append(
            {"instrument": instrument, "side": "buy", "qty": delta, "p
        )
    elif delta < 0:
        orders.append(
            {"instrument": instrument, "side": "sell", "qty": abs(delta)
        )

return orders

def store_snapshot_and_equity(trade_date: str, broker: PaperBroker, prices
cash, gross_market_value, nlv = broker.mark_to_market(prices)
positions = broker.get_positions()

with get_conn() as conn:
    prev = conn.execute(
        "SELECT net_liquidation_value FROM equity_curve WHERE date < ?
        (trade_date, ),
    ).fetchone()

    prev_nlv = float(prev["net_liquidation_value"]) if prev else None
    daily_pnl = None if prev_nlv is None else nlv - prev_nlv
    daily_return = None if prev_nlv in (None, 0) else (nlv / prev_nlv)

    conn.execute(
        """
        INSERT OR REPLACE INTO equity_curve(
            date, cash, gross_market_value, net_liquidation_value, dai
        ) VALUES (?, ?, ?, ?, ?, ?)
        """,
        (trade_date, cash, gross_market_value, nlv, daily_pnl, daily_r
    )

    conn.execute("DELETE FROM portfolio_snapshots WHERE date = ?", (tr

    for instrument, qty in positions.items():

```

```

        if instrument not in prices:
            continue
        close = float(prices[instrument])
        market_value = int(qty) * close
        weight = 0.0 if nlv == 0 else market_value / nlv

        conn.execute(
            """
            INSERT INTO portfolio_snapshots(date, instrument, qty, clo
            VALUES (?, ?, ?, ?, ?, ?)
            """,
            (trade_date, instrument, int(qty), close, market_value, we
        )

def execute_target_positions(target_path: str | Path, prices: dict[str, fl
    broker = PaperBroker()
    targets = pd.read_parquet(target_path)
    trade_date = str(targets["date"].iloc[0])

    orders = generate_orders_from_targets(targets, prices, broker)

    fills = []
    with get_conn() as conn:
        for order in orders:
            conn.execute(
                """
                INSERT INTO orders(date, instrument, side, qty, price, sta
                VALUES (?, ?, ?, ?, ?, ?)
                """,
                (
                    trade_date,
                    order["instrument"],
                    order["side"],
                    int(order["qty"]),
                    float(order["price"]),
                    "created",
                ),
            )

            fill = broker.place_market_order(
                instrument=order["instrument"],
                side=order["side"],
                qty=int(order["qty"]),
                price=float(order["price"]),
            )

            conn.execute(
                """
                INSERT INTO fills(date, instrument, side, qty, price, fee,
                VALUES (?, ?, ?, ?, ?, ?, ?)
                """,

```

```

        (
            trade_date,
            fill.instrument,
            fill.side,
            fill.qty,
            fill.price,
            fill.fee,
            fill.status,
        ),
    )

    fills.append(
        {
            "date": trade_date,
            "instrument": fill.instrument,
            "side": fill.side,
            "qty": fill.qty,
            "price": fill.price,
            "fee": fill.fee,
            "status": fill.status,
        }
    )

    store_snapshot_and_equity(trade_date, broker, prices)

    out = pd.DataFrame(fills)
    out_path = REPORTS_DIR / f"fills_{trade_date}.parquet"
    out.to_parquet(out_path, index=False)
    logger.info(f"Saved fills to {out_path}")
    return out_path

```

run_daily.py

Esto ya usa precios reales.

```

from __future__ import annotations

import os
from dotenv import load_dotenv

from infra.logging_conf import setup_logging
from infra.db import init_db
from research.run_workflow import main as run_workflow
from research.export_signals import export_latest_predictions
from portfolio.rebalance import build_target_positions
from execution.pricing import fetch_and_store_latest_prices
from execution.executor import execute_target_positions

```

```
def run_pipeline():
    load_dotenv()

    run_workflow()

    signal_path = export_latest_predictions(
        experiment_name="backtest_analysis",
        provider_uri=os.getenv("QLIB_PROVIDER_URI", "~/qlib/qlib_data/us_"),
        region=os.getenv("QLIB_REGION", "us"),
    )

    target_path = build_target_positions(signal_path, "conf/strategy.yaml")

    import pandas as pd
    targets = pd.read_parquet(target_path)
    trade_date = str(targets["date"].iloc[0])
    tickers = targets["instrument"].tolist()

    prices = fetch_and_store_latest_prices(trade_date, tickers)
    execute_target_positions(target_path, prices)

if __name__ == "__main__":
    setup_logging()
    init_db()
    run_pipeline()
```

Informe rápido diario

Para ver equity y snapshot del último día:

```
from infra.db import get_conn

with get_conn() as conn:
    eq = conn.execute("""
        SELECT date, cash, gross_market_value, net_liquidation_value, dail
        FROM equity_curve
        ORDER BY date DESC
        LIMIT 5
    """).fetchall()

    for row in eq:
        print(dict(row))

    latest = conn.execute("""
        SELECT MAX(date) AS d FROM portfolio_snapshots
    """).fetchone()["d"]
```

```

snap = conn.execute("""
    SELECT instrument, qty, close, market_value, weight
    FROM portfolio_snapshots
    WHERE date = ?
    ORDER BY market_value DESC
""", (latest,)).fetchall()

for row in snap:
    print(dict(row))

```

Qué cambia de verdad en esta v3

Ya tienes estas piezas cerradas:

- **predicción** con Qlib y su recorder/workflow Qlib Documentati... +2
- **precios reales** desde yfinance o Alpha Vantage; yfinance ofrece download() y Alpha Vantage documenta TIME_SERIES_DAILY_ADJUSTED para closes ajustados ranaroussi.github... +1
- **equity curve persistente**
- **snapshot diario de cartera**
- **PnL diario**
- **cambio fácil de proveedor de mercado**

Lo que haría justo después

Hay cuatro mejoras que sí merecen la pena antes de pensar en live trading real:

1. **slippage configurable**
2. **turnover cap**
3. **reentrenamiento walk-forward**
4. **alertas y health checks**

APScheduler 3.x soporta cron triggers, job stores y persistencia de jobs, así que te sirve para automatizar esta tubería diaria sin demasiada complejidad APScheduler +1

Recomendación práctica

Para arrancar ya:

- usa MARKET_DATA_PROVIDER=yfinance
- prueba el loop 2 o 3 semanas en paper
- revisa cada día equity_curve, fills rechazados, turnover y concentración
- cuando todo cuadre, cambias solo la capa de ejecución por una API de broker

En el siguiente paso te puedo dejar una **v4 con slippage, turnover cap y reporte HTML/CSV diario**.

adelante

Perfecto. Vamos con una **v4** centrada en tres cosas que sí cambian el comportamiento del sistema:

- **slippage configurable**
- **turnover cap**
- **reporte diario HTML + CSV**

La idea es mantener tu arquitectura igual y endurecer la capa de ejecución.

Qué añade esta v4

1. El broker ya no ejecuta exactamente al `close`, sino con un ajuste de slippage.
2. Antes de lanzar órdenes, el sistema calcula el turnover esperado y lo recorta si supera tu límite.
3. Al final del día genera un reporte con:
 - equity
 - PnL diario
 - posiciones
 - fills
 - peso por activo
 - turnover estimado y ejecutado

Cambios de estructura

```
auto_trading_system/  
├─ run_daily.py  
├─ conf/  
│   ├── strategy.yaml  
│   └─ workflow_lightgbm.yaml  
├─ execution/  
│   ├── executor.py  
│   ├── paper_broker.py  
│   ├── pricing.py  
│   └─ turnover.py  
├─ reporting/  
│   └─ daily_report.py  
├─ infra/  
│   └─ db.py
```



```
| | logging_conf.py  
| | paths.py  
...  

```

conf/strategy.yaml

Añade estos parámetros:

```
top_n: 10  
min_score: null  
weight_mode: equal  
max_weight: 0.10  
cash_buffer: 0.02  
  
max_turnover: 0.25  
buy_slippage_bps: 10  
sell_slippage_bps: 10
```

Interpretación:

- `max_turnover: 0.25` → no mover más del 25% del portfolio en un rebalance
- `buy_slippage_bps: 10` → compras 10 bps peor
- `sell_slippage_bps: 10` → ventas 10 bps peor

infra/db.py

Amplía el esquema con métricas de rebalance:

```
import sqlite3  
from contextlib import contextmanager  
from infra.paths import DB_PATH  
  
SCHEMA = """  
CREATE TABLE IF NOT EXISTS signals (  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    score REAL NOT NULL,  
    rank INTEGER,  
    model_version TEXT,  
    PRIMARY KEY (date, instrument, model_version)  
);
```



```
CREATE TABLE IF NOT EXISTS target_positions (  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    target_weight REAL NOT NULL,  
    score REAL,  
    rank INTEGER,  
    PRIMARY KEY (date, instrument)  
);
```

```
CREATE TABLE IF NOT EXISTS orders (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    side TEXT NOT NULL,  
    qty INTEGER NOT NULL,  
    price REAL NOT NULL,  
    status TEXT NOT NULL,  
    created_at TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS fills (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    side TEXT NOT NULL,  
    qty INTEGER NOT NULL,  
    price REAL NOT NULL,  
    fee REAL NOT NULL,  
    status TEXT NOT NULL,  
    created_at TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS broker_state (  
    key TEXT PRIMARY KEY,  
    value TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS daily_prices (  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    close REAL NOT NULL,  
    source TEXT NOT NULL,  
    PRIMARY KEY (date, instrument, source)  
);
```

```
CREATE TABLE IF NOT EXISTS portfolio_snapshots (  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    qty INTEGER NOT NULL,  
    close REAL NOT NULL,  
    market_value REAL NOT NULL,
```

```

        weight REAL NOT NULL,
        PRIMARY KEY (date, instrument)
    );

CREATE TABLE IF NOT EXISTS equity_curve (
    date TEXT PRIMARY KEY,
    cash REAL NOT NULL,
    gross_market_value REAL NOT NULL,
    net_liquidation_value REAL NOT NULL,
    daily_pnl REAL,
    daily_return REAL
);

CREATE TABLE IF NOT EXISTS rebalance_metrics (
    date TEXT PRIMARY KEY,
    portfolio_value REAL NOT NULL,
    target_turnover REAL,
    capped_turnover REAL,
    max_turnover REAL
);
"""

@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()

def init_db() -> None:
    with get_conn() as conn:
        conn.executescript(SCHEMA)

```

execution/turnover.py

Aquí calculamos pesos actuales, turnover objetivo y recorte.

```

from __future__ import annotations

import pandas as pd

```

```

def current_weights_from_positions(
    positions: dict[str, int],
    prices: dict[str, float],
    portfolio_value: float,
) -> pd.DataFrame:
    rows = []
    if portfolio_value <= 0:
        return pd.DataFrame(columns=["instrument", "current_weight"])

    for instrument, qty in positions.items():
        if instrument not in prices:
            continue
        market_value = int(qty) * float(prices[instrument])
        weight = market_value / portfolio_value
        rows.append({"instrument": instrument, "current_weight": weight})

    return pd.DataFrame(rows)

def compute_turnover(current_df: pd.DataFrame, target_df: pd.DataFrame) ->
merged = pd.merge(
    current_df,
    target_df[["instrument", "target_weight"]],
    on="instrument",
    how="outer",
).fillna(0.0)

gross_weight_change = (merged["target_weight"] - merged["current_weight"])
return float(gross_weight_change / 2.0)

def cap_target_weights_by_turnover(
    current_df: pd.DataFrame,
    target_df: pd.DataFrame,
    max_turnover: float,
) -> tuple[pd.DataFrame, float, float]:
    merged = pd.merge(
        current_df,
        target_df[["instrument", "target_weight"]],
        on="instrument",
        how="outer",
    ).fillna(0.0)

    target_turnover = float((merged["target_weight"] - merged["current_weight"]) / 2.0)

    if target_turnover <= max_turnover or target_turnover == 0:
        capped = merged[["instrument", "target_weight"]].copy()
        return capped, target_turnover, target_turnover

    scale = max_turnover / target_turnover
    merged["capped_target_weight"] = (
        merged["current_weight"] + (merged["target_weight"] - merged["current_weight"]) * scale
    )
    return merged, target_turnover, target_turnover

```

```

)

capped = merged[["instrument", "capped_target_weight"]].rename(
    columns={"capped_target_weight": "target_weight"}
)

capped_turnover = float(
    (capped["target_weight"].values - merged["current_weight"].values)
)
capped_turnover = float(abs(capped_turnover).sum() / 2.0)

return capped, target_turnover, capped_turnover

```

execution/paper_broker.py

Añadimos slippage real a la ejecución.

```

from __future__ import annotations

import json
from dataclasses import dataclass
from loguru import logger

from infra.db import get_conn

@dataclass
class Fill:
    instrument: str
    side: str
    qty: int
    price: float
    fee: float
    status: str

class PaperBroker:
    def __init__(
        self,
        initial_cash: float = 100000.0,
        fee_rate: float = 0.001,
        buy_slippage_bps: float = 10.0,
        sell_slippage_bps: float = 10.0,
    ) -> None:
        self.initial_cash = initial_cash

```

```

self.fee_rate = fee_rate
self.buy_slippage_bps = buy_slippage_bps
self.sell_slippage_bps = sell_slippage_bps
self._bootstrap()

def _bootstrap(self) -> None:
    with get_conn() as conn:
        rows = conn.execute("SELECT key, value FROM broker_state").fetchall()
        if not rows:
            conn.execute(
                "INSERT OR REPLACE INTO broker_state(key, value) VALUES (?, ?)"
                ("cash", str(self.initial_cash)),
            )
            conn.execute(
                "INSERT OR REPLACE INTO broker_state(key, value) VALUES (?, ?)"
                ("positions", json.dumps({})),
            )

def get_cash(self) -> float:
    with get_conn() as conn:
        row = conn.execute("SELECT value FROM broker_state WHERE key='cash'").fetchall()
        return float(row[0])

def set_cash(self, cash: float) -> None:
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO broker_state(key, value) VALUES (?, ?)"
            ("cash", str(cash)),
        )

def get_positions(self) -> dict[str, int]:
    with get_conn() as conn:
        row = conn.execute("SELECT value FROM broker_state WHERE key='positions'").fetchall()
        return json.loads(row[0])

def set_positions(self, positions: dict[str, int]) -> None:
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO broker_state(key, value) VALUES (?, ?)"
            ("positions", json.dumps(positions)),
        )

def mark_to_market(self, prices: dict[str, float]) -> tuple[float, float]:
    cash = self.get_cash()
    positions = self.get_positions()
    gross_market_value = 0.0

    for instrument, qty in positions.items():
        if instrument not in prices:
            continue
        gross_market_value += int(qty) * float(prices[instrument])

```

```

nlv = cash + gross_market_value
return cash, gross_market_value, nlv

def _apply_slippage(self, side: str, reference_price: float) -> float:
    if side == "buy":
        return float(reference_price) * (1.0 + self.buy_slippage_bps / 10000)
    if side == "sell":
        return float(reference_price) * (1.0 - self.sell_slippage_bps / 10000)
    raise ValueError("side must be buy or sell")

def place_market_order(self, instrument: str, side: str, qty: int, price: float):
    if qty <= 0:
        raise ValueError("qty must be positive")

    exec_price = self._apply_slippage(side, price)
    cash = self.get_cash()
    positions = self.get_positions()
    gross = qty * exec_price
    fee = gross * self.fee_rate

    if side == "buy":
        total = gross + fee
        if total > cash:
            return Fill(instrument, side, qty, exec_price, fee, "rejected", cash - total)
        positions[instrument] = int(positions.get(instrument, 0)) + qty
        cash -= total

    elif side == "sell":
        current = int(positions.get(instrument, 0))
        if qty > current:
            return Fill(instrument, side, qty, exec_price, fee, "rejected", cash + gross - fee)
        cash += gross - fee
        remaining = current - qty
        if remaining == 0:
            positions.pop(instrument, None)
        else:
            positions[instrument] = remaining

    else:
        raise ValueError("side must be buy/sell")

    self.set_cash(cash)
    self.set_positions(positions)
    logger.info(f"Filled {side} {qty} {instrument} @ {exec_price:.4f}")
    return Fill(instrument, side, qty, exec_price, fee, "filled")

```

Esta es la parte importante. Coge target weights, calcula turnover, los recorta y luego ejecuta.

```
from __future__ import annotations

from pathlib import Path
import pandas as pd
import yaml
from loguru import logger

from execution.paper_broker import PaperBroker
from execution.turnover import (
    current_weights_from_positions,
    compute_turnover,
    cap_target_weights_by_turnover,
)
from infra.db import get_conn
from infra.paths import REPORTS_DIR


def load_strategy_config(config_path: str | Path) -> dict:
    with open(config_path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)


def generate_orders_from_targets(
    targets: pd.DataFrame,
    prices: dict[str, float],
    broker: PaperBroker,
) -> list[dict]:
    _, _, portfolio_value = broker.mark_to_market(prices)
    positions = broker.get_positions()

    target_qty = {}
    for _, row in targets.iterrows():
        instrument = row["instrument"]
        if instrument not in prices:
            continue
        weight = float(row["target_weight"])
        price = float(prices[instrument])
        dollars = portfolio_value * weight
        target_qty[instrument] = int(dollars // price)

    all_instruments = set(positions) | set(target_qty)
    orders = []

    for instrument in sorted(all_instruments):
        current = int(positions.get(instrument, 0))
        target = int(target_qty.get(instrument, 0))
```

```

delta = target - current

if instrument not in prices:
    continue

if delta > 0:
    orders.append(
        {"instrument": instrument, "side": "buy", "qty": delta, "p
    )
elif delta < 0:
    orders.append(
        {"instrument": instrument, "side": "sell", "qty": abs(delta)
    )

return orders

def store_snapshot_and_equity(trade_date: str, broker: PaperBroker, prices: dict,
                             cash, gross_market_value, nlv = broker.mark_to_market(prices),
                             positions = broker.get_positions()):

    with get_conn() as conn:
        prev = conn.execute(
            "SELECT net_liquidation_value FROM equity_curve WHERE date < ?"
            (trade_date, ),
        ).fetchone()

        prev_nlv = float(prev["net_liquidation_value"]) if prev else None
        daily_pnl = None if prev_nlv is None else nlv - prev_nlv
        daily_return = None if prev_nlv in (None, 0) else (nlv / prev_nlv)

        conn.execute(
            """
            INSERT OR REPLACE INTO equity_curve(
                date, cash, gross_market_value, net_liquidation_value, dai
            ) VALUES (?, ?, ?, ?, ?, ?)
            """,
            (trade_date, cash, gross_market_value, nlv, daily_pnl, daily_r
        )

        conn.execute("DELETE FROM portfolio_snapshots WHERE date = ?", (tr

    for instrument, qty in positions.items():
        if instrument not in prices:
            continue
        close = float(prices[instrument])
        market_value = int(qty) * close
        weight = 0.0 if nlv == 0 else market_value / nlv

        conn.execute(
            """
            INSERT INTO portfolio_snapshots(date, instrument, qty, clo

```



```

VALUES (?, ?, ?, ?, ?, ?)
""",
    (trade_date, instrument, int(qty), close, market_value, we
)

def execute_target_positions(
    target_path: str | Path,
    prices: dict[str, float],
    strategy_config_path: str | Path,
) -> Path:
    cfg = load_strategy_config(strategy_config_path)

    broker = PaperBroker(
        buy_slippage_bps=float(cfg.get("buy_slippage_bps", 10)),
        sell_slippage_bps=float(cfg.get("sell_slippage_bps", 10)),
    )

    raw_targets = pd.read_parquet(target_path)
    trade_date = str(raw_targets["date"].iloc[0])

    _, _, portfolio_value = broker.mark_to_market(prices)
    positions = broker.get_positions()
    current_df = current_weights_from_positions(positions, prices, portfol

    capped_weights_df, target_turnover, capped_turnover = cap_target_weigh
        current_df=current_df,
        target_df=raw_targets[["instrument", "target_weight"]],
        max_turnover=float(cfg.get("max_turnover", 1.0)),
    )

    targets = raw_targets.drop(columns=["target_weight"]).merge(
        capped_weights_df,
        on="instrument",
        how="left",
    ).fillna({"target_weight": 0.0})

    orders = generate_orders_from_targets(targets, prices, broker)

    fills = []
    with get_conn() as conn:
        conn.execute(
            """
            INSERT OR REPLACE INTO rebalance_metrics(date, portfolio_value
            VALUES (?, ?, ?, ?, ?)
            """,
            (
                trade_date,
                portfolio_value,
                target_turnover,
                capped_turnover,
                float(cfg.get("max_turnover", 1.0)),

```

```

    ),
)

for order in orders:
    conn.execute(
        """
        INSERT INTO orders(date, instrument, side, qty, price, sta
        VALUES (?, ?, ?, ?, ?, ?)
        """,
        (
            trade_date,
            order["instrument"],
            order["side"],
            int(order["qty"]),
            float(order["price"]),
            "created",
        ),
    )

    fill = broker.place_market_order(
        instrument=order["instrument"],
        side=order["side"],
        qty=int(order["qty"]),
        price=float(order["price"]),
    )

    conn.execute(
        """
        INSERT INTO fills(date, instrument, side, qty, price, fee,
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """,
        (
            trade_date,
            fill.instrument,
            fill.side,
            fill.qty,
            fill.price,
            fill.fee,
            fill.status,
        ),
    )

    fills.append(
        {
            "date": trade_date,
            "instrument": fill.instrument,
            "side": fill.side,
            "qty": fill.qty,
            "price": fill.price,
            "fee": fill.fee,
            "status": fill.status,
        }
    )

```

```

    )

    store_snapshot_and_equity(trade_date, broker, prices)

    out = pd.DataFrame(fills)
    out_path = REPORTS_DIR / f"fills_{trade_date}.parquet"
    out.to_parquet(out_path, index=False)
    logger.info(f"Saved fills to {out_path}")
    return out_path

```

reporting/daily_report.py

Reporte simple pero útil. Genera CSV y HTML.

```

from __future__ import annotations

from pathlib import Path
import pandas as pd

from infra.db import get_conn
from infra.paths import REPORTS_DIR

def generate_daily_report(trade_date: str) -> tuple[Path, Path]:
    with get_conn() as conn:
        equity = pd.read_sql_query(
            """
            SELECT *
            FROM equity_curve
            WHERE date = ?
            """,
            conn,
            params=(trade_date,),
        )

        snapshot = pd.read_sql_query(
            """
            SELECT *
            FROM portfolio_snapshots
            WHERE date = ?
            ORDER BY market_value DESC
            """,
            conn,
            params=(trade_date,),
        )

```

```

fills = pd.read_sql_query(
    """
    SELECT date, instrument, side, qty, price, fee, status
    FROM fills
    WHERE date = ?
    ORDER BY id
    """,
    conn,
    params=(trade_date,),
)

metrics = pd.read_sql_query(
    """
    SELECT *
    FROM rebalance_metrics
    WHERE date = ?
    """,
    conn,
    params=(trade_date,),
)

csv_path = REPORTS_DIR / f"daily_report_{trade_date}.csv"
html_path = REPORTS_DIR / f"daily_report_{trade_date}.html"

snapshot.to_csv(csv_path, index=False)

equity_html = equity.to_html(index=False, border=0)
snapshot_html = snapshot.to_html(index=False, border=0)
fills_html = fills.to_html(index=False, border=0)
metrics_html = metrics.to_html(index=False, border=0)

html = f"""
<html>
<head>
    <meta charset="utf-8">
    <title>Daily Report {trade_date}</title>
    <style>
        body {{ font-family: Arial, sans-serif; margin: 24px; }}
        h1, h2 {{ margin-top: 28px; }}
        table {{ border-collapse: collapse; width: 100%; margin-top: 1
        th, td {{ border: 1px solid #ccc; padding: 8px; font-size: 14p
        th {{ background: #f5f5f5; }}
    </style>
</head>
<body>
    <h1>Daily Trading Report - {trade_date}</h1>
    <h2>Equity</h2>
    {equity_html}
    <h2>Rebalance Metrics</h2>
    {metrics_html}
    <h2>Portfolio Snapshot</h2>

```

```

        {snapshot_html}
        <h2>Fills</h2>
        {fills_html}
    </body>
</html>
"""

html_path.write_text(html, encoding="utf-8")
return csv_path, html_path

```

run_daily.py

Integramos el reporte al final.

```

from __future__ import annotations

import os
from dotenv import load_dotenv
import pandas as pd

from infra.logging_conf import setup_logging
from infra.db import init_db
from research.run_workflow import main as run_workflow
from research.export_signals import export_latest_predictions
from portfolio.rebalance import build_target_positions
from execution.pricing import fetch_and_store_latest_prices
from execution.executor import execute_target_positions
from reporting.daily_report import generate_daily_report

def run_pipeline():
    load_dotenv()

    run_workflow()

    signal_path = export_latest_predictions(
        experiment_name="backtest_analysis",
        provider_uri=os.getenv("QLIB_PROVIDER_URI", "~/qlib/qlib_data/us_"),
        region=os.getenv("QLIB_REGION", "us"),
    )

    target_path = build_target_positions(signal_path, "conf/strategy.yaml")

    targets = pd.read_parquet(target_path)
    trade_date = str(targets["date"].iloc[0])

```

```
tickers = targets["instrument"].tolist()

prices = fetch_and_store_latest_prices(trade_date, tickers)

execute_target_positions(
    target_path=target_path,
    prices=prices,
    strategy_config_path="conf/strategy.yaml",
)

generate_daily_report(trade_date)

if __name__ == "__main__":
    setup_logging()
    init_db()
    run_pipeline()
```

Qué ganas con esta v4

1. Slippage más realista

Sin esto, los backtests y el paper trading suelen salir demasiado optimistas.

2. Control de turnover

Evita que el sistema "persiga" scores pequeños con demasiados cambios de cartera.

3. Reporte operativo

Empiezas a tener una rutina diaria auditable:

- qué quería hacer el sistema
- cuánto movió
- qué ejecutó
- cómo acabó la cartera

Qué revisaría yo cada día

Mira estas cuatro cosas:

- daily_return
- target_turnover vs capped_turnover
- fills rechazados
- concentración de pesos en portfolio_snapshots

Si una estrategia rota demasiado o se concentra de más, suele romper antes en live que en backtest.

Siguiente v5 recomendable

El siguiente salto lógico sería meter:

- **coste por spread dependiente del activo**
- **filtro de liquidez real** con ADV
- **cooldown de reentrada**
- **notificación por Telegram o email**
- **gráfico de equity en el HTML**

La mejora más valiosa de todas sería añadir **walk-forward retraining automático**, porque eso ya te acerca bastante a una operación cuanti seria.

En el siguiente mensaje te dejo una **v5 con gráfico de equity, filtro ADV y alertas Telegram**.



adelante

Perfecto. Vamos con una **v5** que ya se parece bastante a un sistema operativo decente.

Esta versión añade:

- **gráfico de equity** en el HTML
- **filtro ADV** real antes de construir la cartera
- **alertas por Telegram**

La idea es no tocar el núcleo de Qlib, sino endurecer las capas que rodean la señal.

Qué cambia en la v5

1. Filtro ADV

Antes de asignar pesos, el sistema elimina activos con poco volumen negociado.

Usaremos una aproximación simple y útil:

- descargar últimos N días
- calcular $ADV = \text{media}(\text{close} * \text{volume})$
- excluir lo que quede por debajo del umbral

2. Gráfico de equity

El reporte HTML ya no será solo tablas; incluirá una gráfica PNG con la curva de equity.

3. Telegram

Al final del pipeline recibirás un resumen tipo:

- fecha
- equity
- pnl diario
- retorno diario
- nº de fills
- turnover

Estructura nueva

```
auto_trading_system/  
├── reporting/  
│   ├── daily_report.py  
│   └── charts.py  
├── market_data/  
│   ├── providers.py  
│   └── liquidity.py  
├── infra/  
│   ├── notifications.py  
│   └── db.py  
...
```



requirements.txt

Añade estas dependencias:

```
pyqlib  
lightgbm  
pandas  
numpy  
pyyaml  
pyarrow  
loguru
```




```
apscheduler==3.11.0  
yfinance  
requests  
python-dotenv  
matplotlib
```

.env.example

```
MARKET_DATA_PROVIDER=yfinance  
ALPHA_VANTAGE_API_KEY=tu_api_key  
QLIB_PROVIDER_URI=~/.qlib/qlib_data/us_data  
QLIB_REGION=us  
INITIAL_CASH=100000  
FEE_RATE=0.001  
  
TELEGRAM_BOT_TOKEN=tu_bot_token  
TELEGRAM_CHAT_ID=tu_chat_id
```



conf/strategy.yaml

Añade parámetros de liquidez:

```
top_n: 10  
min_score: null  
weight_mode: equal  
max_weight: 0.10  
cash_buffer: 0.02  
  
max_turnover: 0.25  
buy_slippage_bps: 10  
sell_slippage_bps: 10  
  
adv_lookback_days: 20  
min_adv_usd: 5000000
```

market_data/liquidity.py

Este módulo calcula ADV con `yfinance`.


```
from __future__ import annotations

from typing import Iterable
import pandas as pd
import yfinance as yf

def fetch_average_dollar_volume(
    tickers: Iterable[str],
    lookback_days: int = 20,
) -> pd.DataFrame:
    tickers = list(dict.fromkeys(tickers))
    if not tickers:
        return pd.DataFrame(columns=["instrument", "adv_usd"])

    period = f"{max(lookback_days * 2, 30)}d"

    df = yf.download(
        tickers=tickers,
        period=period,
        interval="1d",
        auto_adjust=False,
        progress=False,
        group_by="ticker",
        threads=True,
    )

    rows = []

    if len(tickers) == 1:
        ticker = tickers[0]
        tmp = df.copy()
        tmp = tmp.dropna(subset=["Close", "Volume"]).tail(lookback_days)
        if not tmp.empty:
            adv_usd = float((tmp["Close"] * tmp["Volume"]).mean())
            rows.append({"instrument": ticker, "adv_usd": adv_usd})
        return pd.DataFrame(rows)

    for ticker in tickers:
        if ticker not in df:
            continue
        tmp = df[ticker].dropna(subset=["Close", "Volume"]).tail(lookback_
        if tmp.empty:
            continue
        adv_usd = float((tmp["Close"] * tmp["Volume"]).mean())

</> Python   Ejecutar

    return pd.DataFrame(rows)
```

Cambio en portfolio/rebalance.py

Aquí metemos el filtro ADV antes de seleccionar pesos.

```
from __future__ import annotations

from pathlib import Path
import yaml
import pandas as pd
from loguru import logger

from infra.paths import ORDERS_DIR
from infra.db import get_conn
from portfolio.ranking import select_top_n
from market_data.liquidity import fetch_average_dollar_volume


def load_config(config_path: str | Path) -> dict:
    with open(config_path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)


def apply_adv_filter(signals: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    lookback_days = int(cfg.get("adv_lookback_days", 20))
    min_adv_usd = float(cfg.get("min_adv_usd", 0))

    if min_adv_usd <= 0:
        return signals

    liquidity = fetch_average_dollar_volume(
        signals["instrument"].tolist(),
        lookback_days=lookback_days,
    )

    if liquidity.empty:
        logger.warning("ADV filter returned no liquidity data; keeping original signals")
        return signals

    merged = signals.merge(liquidity, on="instrument", how="left")
    merged["adv_usd"] = merged["adv_usd"].fillna(0.0)

    filtered = merged[merged["adv_usd"] >= min_adv_usd].copy()
    logger.info(f"ADV filter kept {len(filtered)}/{len(signals)} assets")
    return filtered.drop(columns=["adv_usd"])


def build_target_positions(signal_path: str | Path, config_path: str | Path):
    cfg = load_config(config_path)
    signals = pd.read_parquet(signal_path)
```

```

signals = apply_adv_filter(signals, cfg)

selected = select_top_n(
    signals,
    top_n=int(cfg["top_n"]),
    min_score=cfg.get("min_score"),
)

if selected.empty:
    raise ValueError("No assets selected after filters")

cash_buffer = float(cfg.get("cash_buffer", 0.02))
investable = 1.0 - cash_buffer

weight_mode = cfg.get("weight_mode", "equal")
max_weight = float(cfg.get("max_weight", 1.0))

if weight_mode != "equal":
    raise NotImplementedError("Only equal weight is implemented in v5")

base_weight = min(investable / len(selected), max_weight)
selected["target_weight"] = base_weight

remainder = investable - selected["target_weight"].sum()
if remainder > 0:
    selected.loc[selected.index[0], "target_weight"] += remainder

trade_date = selected["date"].iloc[0]
out = selected[["date", "instrument", "target_weight", "score", "rank"]]
out_path = ORDERS_DIR / f"target_positions_{trade_date}.parquet"
out.to_parquet(out_path, index=False)

with get_conn() as conn:
    conn.executemany(
        """
        INSERT OR REPLACE INTO target_positions(date, instrument, targ
        VALUES (?, ?, ?, ?, ?)
        """,
        out.itertuples(index=False, name=None),
    )

logger.info(f"Built {len(out)} target positions for {trade_date}")
return out_path

```

reporting/charts.py

Gráfico de equity sencillo con matplotlib.

```
from __future__ import annotations

from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

from infra.db import get_conn
from infra.paths import REPORTS_DIR

def generate_equity_chart(trade_date: str) -> Path:
    with get_conn() as conn:
        df = pd.read_sql_query(
            """
            SELECT date, net_liquidation_value
            FROM equity_curve
            ORDER BY date
            """,
            conn,
        )

        if df.empty:
            raise ValueError("No equity data available")

        df["date"] = pd.to_datetime(df["date"])

        out_path = REPORTS_DIR / f"equity_curve_{trade_date}.png"

        plt.figure(figsize=(10, 5))
        plt.plot(df["date"], df["net_liquidation_value"])
        plt.title("Equity Curve")
        plt.xlabel("Date")
        plt.ylabel("Net Liquidation Value")
        plt.tight_layout()
        plt.savefig(out_path, dpi=150)
        plt.close()

    return out_path
```

reporting/daily_report.py

Ahora incrustamos la gráfica en el HTML.

```
from __future__ import annotations

from pathlib import Path
import base64
import pandas as pd

from infra.db import get_conn
from infra.paths import REPORTS_DIR
from reporting.charts import generate_equity_chart


def _image_to_base64(path: Path) -> str:
    return base64.b64encode(path.read_bytes()).decode("utf-8")


def generate_daily_report(trade_date: str) -> tuple[Path, Path]:
    with get_conn() as conn:
        equity = pd.read_sql_query(
            "SELECT * FROM equity_curve WHERE date = ?",
            conn,
            params=(trade_date,),
        )

        snapshot = pd.read_sql_query(
            """
            SELECT *
            FROM portfolio_snapshots
            WHERE date = ?
            ORDER BY market_value DESC
            """,
            conn,
            params=(trade_date,),
        )

        fills = pd.read_sql_query(
            """
            SELECT date, instrument, side, qty, price, fee, status
            FROM fills
            WHERE date = ?
            ORDER BY id
            """,
            conn,
            params=(trade_date,),
        )

        metrics = pd.read_sql_query(
            "SELECT * FROM rebalance_metrics WHERE date = ?",
            conn,
            params=(trade_date,),
        )

        csv_path = REPORTS_DIR / f"daily_report_{trade_date}.csv"
```

```
html_path = REPORTS_DIR / f'daily_report_{trade_date}.html'

snapshot.to_csv(csv_path, index=False)

chart_path = generate_equity_chart(trade_date)
chart_b64 = _image_to_base64(chart_path)

html = f"""
<html>
<head>
    <meta charset="utf-8">
    <title>Daily Report {trade_date}</title>
    <style>
        body {{ font-family: Arial, sans-serif; margin: 24px; }}
        h1, h2 {{ margin-top: 28px; }}
        table {{ border-collapse: collapse; width: 100%; margin-top: 1
        th, td {{ border: 1px solid #ccc; padding: 8px; font-size: 14p
        th {{ background: #f5f5f5; }}
        .chart {{ margin-top: 16px; }}
        img {{ max-width: 100%; border: 1px solid #ddd; }}
    </style>
</head>
<body>
    <h1>Daily Trading Report - {trade_date}</h1>

    <h2>Equity</h2>
    {equity.to_html(index=False, border=0)}

    <div class="chart">
        <h2>Equity Curve</h2>
        

    <h2>Rebalance Metrics</h2>
    {metrics.to_html(index=False, border=0)}

    <h2>Portfolio Snapshot</h2>
    {snapshot.to_html(index=False, border=0)}

    <h2>Fills</h2>
    {fills.to_html(index=False, border=0)}
</body>
</html>
"""

html_path.write_text(html, encoding="utf-8")
return csv_path, html_path
```


infra/notifications.py

Cliente Telegram muy simple.

```
from __future__ import annotations

import os
import requests
from loguru import logger

def send_telegram_message(text: str) -> None:
    bot_token = os.getenv("TELEGRAM_BOT_TOKEN")
    chat_id = os.getenv("TELEGRAM_CHAT_ID")

    if not bot_token or not chat_id:
        logger.warning("Telegram credentials not configured; skipping noti")
        return

    url = f"https://api.telegram.org/bot{bot_token}/sendMessage"
    payload = {
        "chat_id": chat_id,
        "text": text,
        "parse_mode": "Markdown",
    }

    resp = requests.post(url, json=payload, timeout=30)
    resp.raise_for_status()
    logger.info("Telegram notification sent")
```

Añade helper en infra/db.py

Para montar un resumen rápido:

```
def fetch_daily_summary(trade_date: str) -> dict:
    with get_conn() as conn:
        eq = conn.execute(
            """
            SELECT cash, gross_market_value, net_liquidation_value, daily_
            FROM equity_curve
            WHERE date = ?
            """,
            (trade_date,),
        )
```

```

    ).fetchone()

    fills_count = conn.execute(
        "SELECT COUNT(*) AS n FROM fills WHERE date = ?",
        (trade_date,),
    ).fetchone()["n"]

    metrics = conn.execute(
        """
        SELECT target_turnover, capped_turnover, max_turnover
        FROM rebalance_metrics
        WHERE date = ?
        """,
        (trade_date,),
    ).fetchone()

    return {
        "date": trade_date,
        "cash": None if eq is None else eq["cash"],
        "gross_market_value": None if eq is None else eq["gross_market_val
        "net_liquidation_value": None if eq is None else eq["net_liquidati
        "daily_pnl": None if eq is None else eq["daily_pnl"],
        "daily_return": None if eq is None else eq["daily_return"],
        "fills_count": fills_count,
        "target_turnover": None if metrics is None else metrics["target_tu
        "capped_turnover": None if metrics is None else metrics["capped_tu
        "max_turnover": None if metrics is None else metrics["max_turnover
    }

```

run_daily.py

Ahora generamos el reporte y enviamos resumen por Telegram.

```

from __future__ import annotations

import os
from dotenv import load_dotenv
import pandas as pd

from infra.logging_conf import setup_logging
from infra.db import init_db, fetch_daily_summary
from infra.notifications import send_telegram_message
from research.run_workflow import main as run_workflow
from research.export_signals import export_latest_predictions
from portfolio.rebalance import build_target_positions

```

```

from execution.pricing import fetch_and_store_latest_prices
from execution.executor import execute_target_positions
from reporting.daily_report import generate_daily_report

def format_daily_summary(summary: dict) -> str:
    daily_return = summary["daily_return"]
    daily_return_pct = "n/a" if daily_return is None else f"{daily_return

    daily_pnl = summary["daily_pnl"]
    daily_pnl_str = "n/a" if daily_pnl is None else f"{daily_pnl:.2f}"

    target_turnover = summary["target_turnover"]
    capped_turnover = summary["capped_turnover"]

    tt = "n/a" if target_turnover is None else f"{target_turnover:.4f}"
    ct = "n/a" if capped_turnover is None else f"{capped_turnover:.4f}"

    return (
        f"*Daily trading summary*\n"
        f>Date: {summary['date']}\n"
        f>NLV: {summary['net_liquidation_value']:.2f}\n"
        f>Daily PnL: {daily_pnl_str}\n"
        f>Daily Return: {daily_return_pct}\n"
        f>Fills: {summary['fills_count']}\n"
        f>Target Turnover: {tt}\n"
        f>Capped Turnover: {ct}"
    )

def run_pipeline():
    load_dotenv()

    run_workflow()

    signal_path = export_latest_predictions(
        experiment_name="backtest_analysis",
        provider_uri=os.getenv("QLIB_PROVIDER_URI", "~/qlib/qlib_data/us_
        region=os.getenv("QLIB_REGION", "us"),
    )

    target_path = build_target_positions(signal_path, "conf/strategy.yaml")

    targets = pd.read_parquet(target_path)
    trade_date = str(targets["date"].iloc[0])
    tickers = targets["instrument"].tolist()

    prices = fetch_and_store_latest_prices(trade_date, tickers)

    execute_target_positions(
        target_path=target_path,
        prices=prices,

```

```
        strategy_config_path="conf/strategy.yaml",
    )

    generate_daily_report(trade_date)

    summary = fetch_daily_summary(trade_date)
    send_telegram_message(format_daily_summary(summary))

if __name__ == "__main__":
    setup_logging()
    init_db()
    run_pipeline()
```

Ajuste recomendable en format_daily_summary

Para evitar errores si NLV viene vacío, usa esta versión:

```
def format_daily_summary(summary: dict) -> str:
    nlv = summary["net_liquidation_value"]
    nlv_str = "n/a" if nlv is None else f"{nlv:.2f}"

    daily_return = summary["daily_return"]
    daily_return_pct = "n/a" if daily_return is None else f"{daily_return

    daily_pnl = summary["daily_pnl"]
    daily_pnl_str = "n/a" if daily_pnl is None else f"{daily_pnl:.2f}"

    target_turnover = summary["target_turnover"]
    capped_turnover = summary["capped_turnover"]

    tt = "n/a" if target_turnover is None else f"{target_turnover:.4f}"
    ct = "n/a" if capped_turnover is None else f"{capped_turnover:.4f}"

    return (
        f"*Daily trading summary*\n"
        f>Date: {summary['date']}\n"
        f>NLV: {nlv_str}\n"
        f>Daily PnL: {daily_pnl_str}\n"
        f>Daily Return: {daily_return_pct}\n"
        f>Fills: {summary['fills_count']}\n"
    )
```

</> Python



▶ Ejecutar

Qué consigues con esta v5

Ya tienes:

- señal de Qlib
- filtro básico de liquidez
- rebalance con turnover cap
- ejecución con slippage
- equity curve
- reporte HTML con gráfico
- alerta operativa por Telegram

Eso ya te permite hacer una fase de **paper trading seria** y revisar el sistema día a día sin mirar la base de datos a mano.

Qué mejoraría en una v6

Lo siguiente que sí merece la pena sería:

- **adjuntar el HTML o PNG al aviso**
- **spread/slippage dependiente del activo**
- **cooldown tras salida**
- **máximo peso por sector**
- **benchmark en el gráfico**
- **comprobaciones de salud** tipo "no se generaron señales", "faltan precios", "demasiados fills rechazados"

La mejora más potente sería meter una **capa de validación pre-trade**, para bloquear automáticamente rebalanceos raros.

Te puedo dejar la **v6 con pre-trade checks, benchmark en la curva de equity y alertas de error**.



adelante

Perfecto. Vamos con una **v6** orientada a robustez operativa.

Añade tres bloques clave:

- **pre-trade checks**
- **benchmark en la curva de equity**
- **alertas de error y estado**

Con esto el sistema deja de ser solo "ejecuta lo que salga" y empieza a comportarse como una tubería controlada.

Qué añade la v6

1. Pre-trade checks

Antes de ejecutar órdenes, el sistema valida cosas como:

- que haya señales
- que haya precios para todos los activos objetivo
- que el número de activos no sea sospechosamente bajo
- que ningún peso exceda un límite duro
- que el turnover no sea absurdo

- que no haya órdenes vacías o tamaños incoherentes

Si algo falla, **no ejecuta** y lanza alerta.

2. Benchmark en el reporte

Además de la equity curve del sistema, el HTML mostrará una curva comparada contra un benchmark, por ejemplo SPY .

3. Alertas de error

El pipeline queda envuelto en `try/except` y notifica por Telegram cuando:

- falla el workflow
- no se pueden descargar precios
- no pasa validación pre-trade
- falla la ejecución
- falta algún artefacto esperado

Estructura nueva

```
auto_trading_system/  
├── run_daily.py  
├── execution/  
│   ├── executor.py  
│   ├── paper_broker.py  
│   ├── pricing.py  
│   ├── turnover.py  
│   └── checks.py  
├── reporting/  
│   ├── daily_report.py  
│   └── charts.py  
├── market_data/  
│   ├── providers.py  
│   └── liquidity.py  
├── infra/  
│   ├── db.py  
│   ├── notifications.py  
│   ├── logging_conf.py  
│   └── paths.py  
└── conf/  
    └── strategy.yaml
```



conf/strategy.yaml

Añade límites duros y benchmark:

```
top_n: 10
min_score: null
weight_mode: equal
max_weight: 0.10
hard_max_weight: 0.15
cash_buffer: 0.02

max_turnover: 0.25
hard_max_turnover: 0.60

buy_slippage_bps: 10
sell_slippage_bps: 10

adv_lookback_days: 20
min_adv_usd: 5000000

min_names: 5
benchmark_ticker: SPY
```

execution/checks.py

Aquí centralizamos las validaciones pre-trade.

```
from __future__ import annotations

from dataclasses import dataclass
import pandas as pd

@dataclass
class CheckResult:
    ok: bool
    messages: list[str]

def run_pretrade_checks(
    targets: pd.DataFrame,
    prices: dict[str, float],
    portfolio_value: float,
    target_turnover: float,
    cfg: dict,
) -> CheckResult:
```



```

messages: list[str] = []

if targets.empty:
    messages.append("Target positions dataframe is empty")

min_names = int(cfg.get("min_names", 1))
if len(targets) < min_names:
    messages.append(f"Too few target names: {len(targets)} < {min_name

missing_prices = sorted(set(targets["instrument"]) - set(prices))
if missing_prices:
    messages.append(f"Missing prices for instruments: {'', '.join(missi

if portfolio_value <= 0:
    messages.append("Portfolio value is non-positive")

if "target_weight" not in targets.columns:
    messages.append("Missing target_weight column")
else:
    hard_max_weight = float(cfg.get("hard_max_weight", 1.0))
    too_large = targets[targets["target_weight"] > hard_max_weight]
    if not too_large.empty:
        messages.append(
            f"Found weights above hard_max_weight={hard_max_weight}: "
            + ", ".join(
                f"{r.instrument}={r.target_weight:.4f}"
                for r in too_large[["instrument", "target_weight"]].it
            )
        )

    neg = targets[targets["target_weight"] < 0]
    if not neg.empty:
        messages.append("Negative target weights found in long-only st

    weight_sum = float(targets["target_weight"].sum())
    if weight_sum > 1.001:
        messages.append(f"Target weights sum above 1.0: {weight_sum:.4

hard_max_turnover = float(cfg.get("hard_max_turnover", 1.0))
if target_turnover > hard_max_turnover:
    messages.append(
        f"Target turnover {target_turnover:.4f} exceeds hard_max_turno
    )

bad_prices = [ticker for ticker, px in prices.items() if px is None or
if bad_prices:
    messages.append(f"Invalid non-positive prices detected: {'', '.join

return CheckResult(ok=(len(messages) == 0), messages=messages)

```

infra/notifications.py

Añadimos helpers para avisos de error y éxito.

```
from __future__ import annotations

import os
import requests
from loguru import logger

def send_telegram_message(text: str) -> None:
    bot_token = os.getenv("TELEGRAM_BOT_TOKEN")
    chat_id = os.getenv("TELEGRAM_CHAT_ID")

    if not bot_token or not chat_id:
        logger.warning("Telegram credentials not configured; skipping noti")
        return

    url = f"https://api.telegram.org/bot{bot_token}/sendMessage"
    payload = {
        "chat_id": chat_id,
        "text": text,
        "parse_mode": "Markdown",
    }

    resp = requests.post(url, json=payload, timeout=30)
    resp.raise_for_status()
    logger.info("Telegram notification sent")

def send_error_alert(stage: str, error: str) -> None:
    msg = (
        f"*Trading pipeline error*\n"
        f"Stage: {stage}\n"
        f"Error: {error}"
    )
    send_telegram_message(msg)

def send_success_alert(text: str) -> None:
    send_telegram_message(text)
```

infra/db.py

Añadimos benchmark diario.

```
import sqlite3
from contextlib import contextmanager
from infra.paths import DB_PATH

SCHEMA = """
CREATE TABLE IF NOT EXISTS signals (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    score REAL NOT NULL,
    rank INTEGER,
    model_version TEXT,
    PRIMARY KEY (date, instrument, model_version)
);

CREATE TABLE IF NOT EXISTS target_positions (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    target_weight REAL NOT NULL,
    score REAL,
    rank INTEGER,
    PRIMARY KEY (date, instrument)
);

CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    side TEXT NOT NULL,
    qty INTEGER NOT NULL,
    price REAL NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS fills (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    side TEXT NOT NULL,
    qty INTEGER NOT NULL,
    price REAL NOT NULL,
    fee REAL NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE IF NOT EXISTS broker_state (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS daily_prices (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    close REAL NOT NULL,
    source TEXT NOT NULL,
    PRIMARY KEY (date, instrument, source)
);

CREATE TABLE IF NOT EXISTS portfolio_snapshots (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    qty INTEGER NOT NULL,
    close REAL NOT NULL,
    market_value REAL NOT NULL,
    weight REAL NOT NULL,
    PRIMARY KEY (date, instrument)
);

CREATE TABLE IF NOT EXISTS equity_curve (
    date TEXT PRIMARY KEY,
    cash REAL NOT NULL,
    gross_market_value REAL NOT NULL,
    net_liquidation_value REAL NOT NULL,
    daily_pnl REAL,
    daily_return REAL
);

CREATE TABLE IF NOT EXISTS rebalance_metrics (
    date TEXT PRIMARY KEY,
    portfolio_value REAL NOT NULL,
    target_turnover REAL,
    capped_turnover REAL,
    max_turnover REAL
);

CREATE TABLE IF NOT EXISTS benchmark_prices (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    close REAL NOT NULL,
    PRIMARY KEY (date, instrument)
);
""""
```

@contextmanager

```

def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()

def init_db() -> None:
    with get_conn() as conn:
        conn.executescript(SCHEMA)

def fetch_daily_summary(trade_date: str) -> dict:
    with get_conn() as conn:
        eq = conn.execute(
            """
            SELECT cash, gross_market_value, net_liquidation_value, daily_
            FROM equity_curve
            WHERE date = ?
            """,
            (trade_date,),
        ).fetchone()

        fills_count = conn.execute(
            "SELECT COUNT(*) AS n FROM fills WHERE date = ?",
            (trade_date,),
        ).fetchone()["n"]

        metrics = conn.execute(
            """
            SELECT target_turnover, capped_turnover, max_turnover
            FROM rebalance_metrics
            WHERE date = ?
            """,
            (trade_date,),
        ).fetchone()

    return {
        "date": trade_date,
        "cash": None if eq is None else eq["cash"],
        "gross_market_value": None if eq is None else eq["gross_market_val
        "net_liquidation_value": None if eq is None else eq["net_liquidati
        "daily_pnl": None if eq is None else eq["daily_pnl"],
        "daily_return": None if eq is None else eq["daily_return"],
        "fills_count": fills_count,
        "target_turnover": None if metrics is None else metrics["target_tu
        "capped_turnover": None if metrics is None else metrics["capped_tu

```

```
"max_turnover": None if metrics is None else metrics["max_turnover"]
}
```

execution/pricing.py

Guardamos también benchmark.

```
from __future__ import annotations

from loguru import logger
from infra.db import get_conn
from market_data.providers import build_price_provider

def fetch_and_store_latest_prices(trade_date: str, tickers: list[str]) ->
    provider = build_price_provider()
    prices = provider.get_latest_closes(tickers)

    if not prices:
        raise ValueError("No prices fetched")

    with get_conn() as conn:
        conn.executemany(
            """
            INSERT OR REPLACE INTO daily_prices(date, instrument, close, s
            VALUES (?, ?, ?, ?)
            """,
            [(trade_date, ticker, float(price), provider.source_name) for
            )

    logger.info(f"Stored {len(prices)} prices for {trade_date} from {provi
    return prices

def fetch_and_store_benchmark_price(trade_date: str, benchmark_ticker: str
    provider = build_price_provider()
    prices = provider.get_latest_closes([benchmark_ticker])

    if benchmark_ticker not in prices:
        raise ValueError(f"Missing benchmark price for {benchmark_ticker}")

    close = float(prices[benchmark_ticker])

    with get_conn() as conn:
        conn.execute(
```

```

        """
        INSERT OR REPLACE INTO benchmark_prices(date, instrument, close)
        VALUES (?, ?, ?)
        """
        (trade_date, benchmark_ticker, close),
    )

    logger.info(f"Stored benchmark price {benchmark_ticker}={close:.4f} for {trade_date}")
    return close

```

reporting/charts.py

Ahora genera una curva comparada contra benchmark normalizado a 100.

```

from __future__ import annotations

from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

from infra.db import get_conn
from infra.paths import REPORTS_DIR

def generate_equity_chart(trade_date: str) -> Path:
    with get_conn() as conn:
        eq = pd.read_sql_query(
            """
            SELECT date, net_liquidation_value
            FROM equity_curve
            ORDER BY date
            """,
            conn,
        )

        bm = pd.read_sql_query(
            """
            SELECT date, close
            FROM benchmark_prices
            ORDER BY date
            """,
            conn,
        )

        if eq.empty:

```

```

        raise ValueError("No equity data available")

eq["date"] = pd.to_datetime(eq["date"])
eq["equity_index"] = eq["net_liquidation_value"] / eq["net_liquidation"]

out_path = REPORTS_DIR / f"equity_curve_{trade_date}.png"

plt.figure(figsize=(10, 5))
plt.plot(eq["date"], eq["equity_index"], label="Strategy")

if not bm.empty:
    bm["date"] = pd.to_datetime(bm["date"])
    bm["benchmark_index"] = bm["close"] / bm["close"].iloc[0] * 100.0
    plt.plot(bm["date"], bm["benchmark_index"], label="Benchmark")

plt.title("Strategy vs Benchmark (Base 100)")
plt.xlabel("Date")
plt.ylabel("Indexed Value")
plt.legend()
plt.tight_layout()
plt.savefig(out_path, dpi=150)
plt.close()

return out_path

```

execution/executor.py

Aquí metemos el check antes de ejecutar.

```

from __future__ import annotations

from pathlib import Path
import pandas as pd
import yaml
from loguru import logger

from execution.paper_broker import PaperBroker
from execution.turnover import current_weights_from_positions, cap_target_
from execution.checks import run_pretrade_checks
from infra.db import get_conn
from infra.paths import REPORTS_DIR

def load_strategy_config(config_path: str | Path) -> dict:
    with open(config_path, "r", encoding="utf-8") as f:

```



```

        return yaml.safe_load(f)

def generate_orders_from_targets(
    targets: pd.DataFrame,
    prices: dict[str, float],
    broker: PaperBroker,
) -> list[dict]:
    _, _, portfolio_value = broker.mark_to_market(prices)
    positions = broker.get_positions()

    target_qty = {}
    for _, row in targets.iterrows():
        instrument = row["instrument"]
        if instrument not in prices:
            continue
        weight = float(row["target_weight"])
        price = float(prices[instrument])
        dollars = portfolio_value * weight
        target_qty[instrument] = int(dollars // price)

    all_instruments = set(positions) | set(target_qty)
    orders = []

    for instrument in sorted(all_instruments):
        current = int(positions.get(instrument, 0))
        target = int(target_qty.get(instrument, 0))
        delta = target - current

        if instrument not in prices:
            continue

        if delta > 0:
            orders.append(
                {"instrument": instrument, "side": "buy", "qty": delta, "p
            )
        elif delta < 0:
            orders.append(
                {"instrument": instrument, "side": "sell", "qty": abs(delta
            )

    return orders

def store_snapshot_and_equity(trade_date: str, broker: PaperBroker, prices
cash, gross_market_value, nlv = broker.mark_to_market(prices)
positions = broker.get_positions()

with get_conn() as conn:
    prev = conn.execute(
        "SELECT net_liquidation_value FROM equity_curve WHERE date < ?
        (trade_date,),

```

```

).fetchone()

prev_nlv = float(prev["net_liquidation_value"]) if prev else None
daily_pnl = None if prev_nlv is None else nlv - prev_nlv
daily_return = None if prev_nlv in (None, 0) else (nlv / prev_nlv)

conn.execute(
    """
    INSERT OR REPLACE INTO equity_curve(
        date, cash, gross_market_value, net_liquidation_value, dai
    ) VALUES (?, ?, ?, ?, ?, ?)
    """,
    (trade_date, cash, gross_market_value, nlv, daily_pnl, daily_r
)

conn.execute("DELETE FROM portfolio_snapshots WHERE date = ?", (tr

for instrument, qty in positions.items():
    if instrument not in prices:
        continue
    close = float(prices[instrument])
    market_value = int(qty) * close
    weight = 0.0 if nlv == 0 else market_value / nlv

    conn.execute(
        """
        INSERT INTO portfolio_snapshots(date, instrument, qty, clo
        VALUES (?, ?, ?, ?, ?, ?)
        """,
        (trade_date, instrument, int(qty), close, market_value, we
    )

def execute_target_positions(
    target_path: str | Path,
    prices: dict[str, float],
    strategy_config_path: str | Path,
) -> Path:
    cfg = load_strategy_config(strategy_config_path)

    broker = PaperBroker(
        buy_slippage_bps=float(cfg.get("buy_slippage_bps", 10)),
        sell_slippage_bps=float(cfg.get("sell_slippage_bps", 10)),
    )

    raw_targets = pd.read_parquet(target_path)
    trade_date = str(raw_targets["date"].iloc[0])

    _, _, portfolio_value = broker.mark_to_market(prices)
    positions = broker.get_positions()
    current_df = current_weights_from_positions(positions, prices, portfol

```

```

capped_weights_df, target_turnover, capped_turnover = cap_target_weight(
    current_df=current_df,
    target_df=raw_targets[["instrument", "target_weight"]],
    max_turnover=float(cfg.get("max_turnover", 1.0)),
)

targets = raw_targets.drop(columns=["target_weight"]).merge(
    capped_weights_df,
    on="instrument",
    how="left",
).fillna({"target_weight": 0.0})

check = run_pretrade_checks(
    targets=targets,
    prices=prices,
    portfolio_value=portfolio_value,
    target_turnover=target_turnover,
    cfg=cfg,
)

if not check.ok:
    raise RuntimeError("Pre-trade checks failed: " + " | ".join(check.

orders = generate_orders_from_targets(targets, prices, broker)

fills = []
with get_conn() as conn:
    conn.execute(
        """
        INSERT OR REPLACE INTO rebalance_metrics(date, portfolio_value
        VALUES (?, ?, ?, ?, ?)
        """,
        (
            trade_date,
            portfolio_value,
            target_turnover,
            capped_turnover,
            float(cfg.get("max_turnover", 1.0)),
        ),
    )

for order in orders:
    conn.execute(
        """
        INSERT INTO orders(date, instrument, side, qty, price, sta
        VALUES (?, ?, ?, ?, ?, ?)
        """,
        (
            trade_date,
            order["instrument"],
            order["side"],
            int(order["qty"]),
            float(order["price"]),

```

```

        "created",
    ),
)

fill = broker.place_market_order(
    instrument=order["instrument"],
    side=order["side"],
    qty=int(order["qty"]),
    price=float(order["price"]),
)

conn.execute(
    """
    INSERT INTO fills(date, instrument, side, qty, price, fee,
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """,
    (
        trade_date,
        fill.instrument,
        fill.side,
        fill.qty,
        fill.price,
        fill.fee,
        fill.status,
    ),
)

fills.append(
    {
        "date": trade_date,
        "instrument": fill.instrument,
        "side": fill.side,
        "qty": fill.qty,
        "price": fill.price,
        "fee": fill.fee,
        "status": fill.status,
    }
)

store_snapshot_and_equity(trade_date, broker, prices)

out = pd.DataFrame(fills)
out_path = REPORTS_DIR / f"fills_{trade_date}.parquet"
out.to_parquet(out_path, index=False)
logger.info(f"Saved fills to {out_path}")
return out_path

```

run_daily.py

Ahora el pipeline tiene control de errores por etapa.

```
from __future__ import annotations

import os
from dotenv import load_dotenv
import pandas as pd
from loguru import logger

from infra.logging_conf import setup_logging
from infra.db import init_db, fetch_daily_summary
from infra.notifications import send_error_alert, send_success_alert
from research.run_workflow import main as run_workflow
from research.export_signals import export_latest_predictions
from portfolio.rebalance import build_target_positions
from execution.pricing import fetch_and_store_latest_prices, fetch_and_store_prices
from execution.executor import execute_target_positions
from reporting.daily_report import generate_daily_report


def format_daily_summary(summary: dict) -> str:
    nlv = summary["net_liquidation_value"]
    nlv_str = "n/a" if nlv is None else f"{nlv:.2f}"

    daily_return = summary["daily_return"]
    daily_return_pct = "n/a" if daily_return is None else f"{daily_return:.2f}"

    daily_pnl = summary["daily_pnl"]
    daily_pnl_str = "n/a" if daily_pnl is None else f"{daily_pnl:.2f}"

    target_turnover = summary["target_turnover"]
    capped_turnover = summary["capped_turnover"]

    tt = "n/a" if target_turnover is None else f"{target_turnover:.4f}"
    ct = "n/a" if capped_turnover is None else f"{capped_turnover:.4f}"

    return (
        f"*Daily trading summary*\n"
        f>Date: {summary['date']}\n"
        f>NLV: {nlv_str}\n"
        f>Daily PnL: {daily_pnl_str}\n"
        f>Daily Return: {daily_return_pct}\n"
        f>Fills: {summary['fills_count']}\n"
        f>Target Turnover: {tt}\n"
        f>Capped Turnover: {ct}"
    )
```

```
def run_pipeline():
    load_dotenv()

    try:
        run_workflow()
    except Exception as e:
        send_error_alert("workflow", str(e))
        raise

    try:
        signal_path = export_latest_predictions(
            experiment_name="backtest_analysis",
            provider_uri=os.getenv("QLIB_PROVIDER_URI", "~/.qlib/qlib_data"),
            region=os.getenv("QLIB_REGION", "us"),
        )
    except Exception as e:
        send_error_alert("export_signals", str(e))
        raise

    try:
        target_path = build_target_positions(signal_path, "conf/strategy.y
        targets = pd.read_parquet(target_path)
        trade_date = str(targets["date"].iloc[0])
        tickers = targets["instrument"].tolist()
    except Exception as e:
        send_error_alert("build_targets", str(e))
        raise

    try:
        prices = fetch_and_store_latest_prices(trade_date, tickers)
    except Exception as e:
        send_error_alert("fetch_prices", str(e))
        raise

    try:
        import yaml
        with open("conf/strategy.yaml", "r", encoding="utf-8") as f:
            cfg = yaml.safe_load(f)
            benchmark_ticker = cfg.get("benchmark_ticker", "SPY")
            fetch_and_store_benchmark_price(trade_date, benchmark_ticker)
    except Exception as e:
        send_error_alert("benchmark_price", str(e))
        raise

    try:
        execute_target_positions(
            target_path=target_path,
            prices=prices,
            strategy_config_path="conf/strategy.yaml",
        )
    except Exception as e:
```

```
        send_error_alert("execution", str(e))
        raise

    try:
        generate_daily_report(trade_date)
        summary = fetch_daily_summary(trade_date)
        send_success_alert(format_daily_summary(summary))
    except Exception as e:
        send_error_alert("reporting", str(e))
        raise

if __name__ == "__main__":
    setup_logging()
    init_db()

    try:
        run_pipeline()
    except Exception:
        logger.exception("Pipeline failed")
        raise
```

Qué consigues con esta v6

Ahora ya tienes un sistema que:

- no ejecuta a ciegas
- compara contra benchmark
- te avisa cuando algo falla
- deja trazabilidad diaria más útil

Eso es bastante más importante de lo que parece. Muchos sistemas “funcionan” hasta que un día faltan precios, salen dos señales en vez de diez o el rebalance intenta mover media cartera sin control.

Qué vigilaría yo a partir de aquí

Las siguientes señales de alarma:

- target names muy por debajo de lo normal
- turnover objetivo demasiado alto varios días seguidos
- demasiados fills rechazados
- equity peor que benchmark durante periodos prolongados
- dependencia excesiva de 2 o 3 nombres

v7 lógica

El siguiente salto útil no sería añadir más adornos, sino **control de riesgo de verdad**:

- límite por sector
- límite por correlación
- stop de portfolio
- cool-down tras salida
- suspensión automática tras drawdown
- validación walk-forward automática

La pieza más valiosa sería una **capa de risk manager previa a execution**.

Te la puedo dejar en una **v7 con risk manager, max drawdown guard y límites por sector**.



adelante

Perfecto. Vamos con una **v7** centrada en una pieza que sí cambia el perfil del sistema: un **risk manager** entre la señal y la ejecución.

La idea es esta:

Qlib propone → portfolio construye pesos → risk manager los recorta o bloquea → executor ejecuta.

Eso te permite poner límites globales sin contaminar el modelo.

Qué añade la v7

Tres bloques:

- límite por sector
- guard de max drawdown
- cooldown tras salida

Qué resuelve cada uno

Límite por sector

Evita que una cartera aparentemente "diversificada" esté en realidad concentrada en tecnología, semis, bancos, etc.

Max drawdown guard

Si la curva de equity cae demasiado desde su máximo, el sistema entra en modo defensivo: no rebalancea, o rebalancea a cash parcial/total.

Cooldown tras salida

Si un activo acaba de salir de cartera, no lo dejas reentrar inmediatamente durante X días. Esto reduce churn y ruido.

Estructura nueva

```
auto_trading_system/  
├── execution/  
│   ├── checks.py  
│   ├── executor.py  
│   ├── paper_broker.py  
│   ├── turnover.py  
│   └── risk_manager.py  
├── market_data/  
│   └── metadata.py  
├── infra/  
│   └── db.py  
└── conf/  
    └── strategy.yaml
```



conf/strategy.yaml

Añade esto:

```
top_n: 10  
min_score: null  
weight_mode: equal  
max_weight: 0.10  
hard_max_weight: 0.15  
cash_buffer: 0.02  
  
max_turnover: 0.25  
hard_max_turnover: 0.60  
  
buy_slippage_bps: 10  
sell_slippage_bps: 10  
  
adv_lookback_days: 20
```

```
min_adv_usd: 5000000

min_names: 5
benchmark_ticker: SPY

max_sector_weight: 0.35
max_portfolio_drawdown: 0.12
drawdown_action: reduce_to_cash
drawdown_cash_target: 1.00

cooldown_days_after_exit: 5
```

Interpretación:

- `max_sector_weight: 0.35` → ningún sector puede pasar del 35%
- `max_portfolio_drawdown: 0.12` → si caes 12% desde pico, se activa guard
- `drawdown_action: reduce_to_cash` → la cartera objetivo se vacía
- `cooldown_days_after_exit: 5` → no recomprar un nombre hasta 5 días después de salir

infra/db.py

Añade tablas para metadatos, riesgo y cooldown.

```
import sqlite3
from contextlib import contextmanager
from infra.paths import DB_PATH

SCHEMA = """
CREATE TABLE IF NOT EXISTS signals (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    score REAL NOT NULL,
    rank INTEGER,
    model_version TEXT,
    PRIMARY KEY (date, instrument, model_version)
);

CREATE TABLE IF NOT EXISTS target_positions (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    target_weight REAL NOT NULL,
    score REAL,
    rank INTEGER,
    PRIMARY KEY (date, instrument)
```

```
);
```

```
CREATE TABLE IF NOT EXISTS orders (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    side TEXT NOT NULL,  
    qty INTEGER NOT NULL,  
    price REAL NOT NULL,  
    status TEXT NOT NULL,  
    created_at TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS fills (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    side TEXT NOT NULL,  
    qty INTEGER NOT NULL,  
    price REAL NOT NULL,  
    fee REAL NOT NULL,  
    status TEXT NOT NULL,  
    created_at TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS broker_state (  
    key TEXT PRIMARY KEY,  
    value TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS daily_prices (  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    close REAL NOT NULL,  
    source TEXT NOT NULL,  
    PRIMARY KEY (date, instrument, source)  
);
```

```
CREATE TABLE IF NOT EXISTS portfolio_snapshots (  
    date TEXT NOT NULL,  
    instrument TEXT NOT NULL,  
    qty INTEGER NOT NULL,  
    close REAL NOT NULL,  
    market_value REAL NOT NULL,  
    weight REAL NOT NULL,  
    PRIMARY KEY (date, instrument)  
);
```

```
CREATE TABLE IF NOT EXISTS equity_curve (  
    date TEXT PRIMARY KEY,  
    cash REAL NOT NULL,  
    gross_market_value REAL NOT NULL,
```

```

        net_liquidation_value REAL NOT NULL,
        daily_pnl REAL,
        daily_return REAL
    );

CREATE TABLE IF NOT EXISTS rebalance_metrics (
    date TEXT PRIMARY KEY,
    portfolio_value REAL NOT NULL,
    target_turnover REAL,
    capped_turnover REAL,
    max_turnover REAL
);

CREATE TABLE IF NOT EXISTS benchmark_prices (
    date TEXT NOT NULL,
    instrument TEXT NOT NULL,
    close REAL NOT NULL,
    PRIMARY KEY (date, instrument)
);

CREATE TABLE IF NOT EXISTS instrument_metadata (
    instrument TEXT PRIMARY KEY,
    sector TEXT
);

CREATE TABLE IF NOT EXISTS risk_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    event_type TEXT NOT NULL,
    message TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS instrument_cooldown (
    instrument TEXT PRIMARY KEY,
    cooldown_until TEXT NOT NULL,
    reason TEXT
);
"""

@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()

```

```

def init_db() -> None:
    with get_conn() as conn:
        conn.executescript(SCHEMA)

def fetch_daily_summary(trade_date: str) -> dict:
    with get_conn() as conn:
        eq = conn.execute(
            """
            SELECT cash, gross_market_value, net_liquidation_value, daily_
            FROM equity_curve
            WHERE date = ?
            """,
            (trade_date,),
        ).fetchone()

        fills_count = conn.execute(
            "SELECT COUNT(*) AS n FROM fills WHERE date = ?",
            (trade_date,),
        ).fetchone()["n"]

        metrics = conn.execute(
            """
            SELECT target_turnover, capped_turnover, max_turnover
            FROM rebalance_metrics
            WHERE date = ?
            """,
            (trade_date,),
        ).fetchone()

        risk_count = conn.execute(
            "SELECT COUNT(*) AS n FROM risk_events WHERE date = ?",
            (trade_date,),
        ).fetchone()["n"]

    return {
        "date": trade_date,
        "cash": None if eq is None else eq["cash"],
        "gross_market_value": None if eq is None else eq["gross_market_val
        "net_liquidation_value": None if eq is None else eq["net_liquidati
        "daily_pnl": None if eq is None else eq["daily_pnl"],
        "daily_return": None if eq is None else eq["daily_return"],
        "fills_count": fills_count,
        "risk_events_count": risk_count,
        "target_turnover": None if metrics is None else metrics["target_tu
        "capped_turnover": None if metrics is None else metrics["capped_tu
        "max_turnover": None if metrics is None else metrics["max_turnover
    }

```

market_data/metadata.py

Versión mínima: mapa manual de sectores. Más adelante lo puedes reemplazar por proveedor real.

```
from __future__ import annotations

from infra.db import get_conn

DEFAULT_SECTORS = {
    "AAPL": "Technology",
    "MSFT": "Technology",
    "NVDA": "Technology",
    "AMD": "Technology",
    "AVGO": "Technology",
    "META": "Communication Services",
    "GOOGL": "Communication Services",
    "NFLX": "Communication Services",
    "AMZN": "Consumer Discretionary",
    "TSLA": "Consumer Discretionary",
    "ADBE": "Technology",
    "SPY": "ETF",
}

def seed_default_instrument_metadata() -> None:
    with get_conn() as conn:
        conn.executemany(
            """
            INSERT OR REPLACE INTO instrument_metadata(instrument, sector)
            VALUES (?, ?)
            """,
            [(k, v) for k, v in DEFAULT_SECTORS.items()],
        )

def get_sector_map(instruments: list[str]) -> dict[str, str]:
    if not instruments:
        return {}

    placeholders = ",".join("?" for _ in instruments)
    with get_conn() as conn:
        rows = conn.execute(
            f"""
            SELECT instrument, sector
            FROM instrument_metadata
            WHERE instrument IN ({placeholders})
            """,
        )
```

```

        instruments,
    ).fetchall()

    sector_map = {row["instrument"]: row["sector"] for row in rows}
    for instrument in instruments:
        sector_map.setdefault(instrument, "Unknown")
    return sector_map

```

execution/risk_manager.py

Aquí va la lógica principal.

```

from __future__ import annotations

import pandas as pd

from infra.db import get_conn
from market_data.metadata import get_sector_map

def record_risk_event(trade_date: str, event_type: str, message: str) -> None:
    with get_conn() as conn:
        conn.execute(
            """
            INSERT INTO risk_events(date, event_type, message)
            VALUES (?, ?, ?)
            """,
            (trade_date, event_type, message),
        )

def get_peak_equity_and_drawdown(current_nlv: float) -> tuple[float | None]:
    with get_conn() as conn:
        row = conn.execute(
            """
            SELECT MAX(net_liquidation_value) AS peak_nlv
            FROM equity_curve
            """
        ).fetchone()

        peak_nlv = None if row is None else row["peak_nlv"]
        if peak_nlv in (None, 0):
            return peak_nlv, None

        drawdown = 1.0 - (current_nlv / float(peak_nlv))

```

```

    return float(peak_nlv), float(drawdown)

def apply_drawdown_guard(
    trade_date: str,
    targets: pd.DataFrame,
    current_nlv: float,
    cfg: dict,
) -> pd.DataFrame:
    max_dd = float(cfg.get("max_portfolio_drawdown", 1.0))
    action = cfg.get("drawdown_action", "none")
    cash_target = float(cfg.get("drawdown_cash_target", 1.0))

    peak_nlv, drawdown = get_peak_equity_and_drawdown(current_nlv)
    if drawdown is None or drawdown <= max_dd:
        return targets

    msg = f"Drawdown guard triggered: drawdown={drawdown:.4f}, limit={max_
record_risk_event(trade_date, "drawdown_guard", msg)

    if action == "reduce_to_cash":
        out = targets.copy()
        out["target_weight"] = 0.0
        return out

    if action == "scale_down":
        scale = max(0.0, 1.0 - cash_target)
        out = targets.copy()
        out["target_weight"] = out["target_weight"] * scale
        return out

    return targets

def apply_sector_caps(
    trade_date: str,
    targets: pd.DataFrame,
    cfg: dict,
) -> pd.DataFrame:
    max_sector_weight = float(cfg.get("max_sector_weight", 1.0))
    if targets.empty:
        return targets

    out = targets.copy()
    sector_map = get_sector_map(out["instrument"].tolist())
    out["sector"] = out["instrument"].map(sector_map)

    sector_weights = out.groupby("sector", as_index=False)["target_weight"]

    for row in sector_weights.itertuples(index=False):
        if row.target_weight <= max_sector_weight:
            continue

```



```

sector_mask = out["sector"] == row.sector
scale = max_sector_weight / float(row.target_weight)
out.loc[sector_mask, "target_weight"] = out.loc[sector_mask, "target_weight"] * scale

msg = (
    f"Sector cap applied: sector={row.sector}, "
    f"raw_weight={row.target_weight:.4f}, max={max_sector_weight:.4f}"
)
record_risk_event(trade_date, "sector_cap", msg)

return out.drop(columns=["sector"])

def update_cooldown_registry(
    trade_date: str,
    previous_positions: dict[str, int],
    target_weights: pd.DataFrame,
    cooldown_days: int,
) -> None:
    if cooldown_days <= 0:
        return

    prev_names = {k for k, v in previous_positions.items() if int(v) > 0}
    next_names = set(target_weights[target_weights["target_weight"] > 0]["instrument"])
    exited = sorted(prev_names - next_names)

    if not exited:
        return

    trade_dt = pd.Timestamp(trade_date)
    cooldown_until = (trade_dt + pd.Timedelta(days=cooldown_days)).strftime("%Y-%m-%d")

    with get_conn() as conn:
        for instrument in exited:
            conn.execute(
                """
                INSERT OR REPLACE INTO instrument_cooldown(instrument, cooldown_until, recent_exit)
                VALUES (?, ?, ?)
                """,
                (instrument, cooldown_until, "recent_exit"),
            )
            record_risk_event(
                trade_date,
                "cooldown_set",
                f"Cooldown set for {instrument} until {cooldown_until}",
            )

def apply_cooldown_filter(
    trade_date: str,
    targets: pd.DataFrame,

```

```

) -> pd.DataFrame:
    if targets.empty:
        return targets

    with get_conn() as conn:
        rows = conn.execute(
            """
            SELECT instrument, cooldown_until
            FROM instrument_cooldown
            WHERE cooldown_until >= ?
            """,
            (trade_date,),
        ).fetchall()

    blocked = {row["instrument"] for row in rows}
    if not blocked:
        return targets

    out = targets.copy()
    mask = out["instrument"].isin(blocked) & (out["target_weight"] > 0)
    blocked_names = out.loc[mask, "instrument"].tolist()

    if blocked_names:
        out.loc[mask, "target_weight"] = 0.0
        record_risk_event(
            trade_date,
            "cooldown_filter",
            f"Blocked by cooldown: {'', ' '.join(sorted(blocked_names))}",
        )

    return out

def renormalize_weights(targets: pd.DataFrame, max_total_weight: float = 0)
    out = targets.copy()
    total = float(out["target_weight"].sum())
    if total <= 0:
        return out

    if total > max_total_weight:
        out["target_weight"] = out["target_weight"] * (max_total_weight /

    return out

def apply_risk_manager(
    trade_date: str,
    targets: pd.DataFrame,
    current_nlv: float,
    previous_positions: dict[str, int],
    cfg: dict,
) -> pd.DataFrame:

```

```

out = targets.copy()

out = apply_drawdown_guard(
    trade_date=trade_date,
    targets=out,
    current_nlv=current_nlv,
    cfg=cfg,
)

out = apply_sector_caps(
    trade_date=trade_date,
    targets=out,
    cfg=cfg,
)

out = apply_cooldown_filter(
    trade_date=trade_date,
    targets=out,
)

out = renormalize_weights(out, max_total_weight=1.0 - float(cfg.get("c

cooldown_days = int(cfg.get("cooldown_days_after_exit", 0))
update_cooldown_registry(
    trade_date=trade_date,
    previous_positions=previous_positions,
    target_weights=out,
    cooldown_days=cooldown_days,
)

return out

```

Cambio en execution/executor.py

Solo te marco la parte nueva para no repetir todo el archivo.

Importa el risk manager

```
from execution.risk_manager import apply_risk_manager
```

Dentro de execute_target_positions(...), justo después del turnover cap

Sustituye esta parte:

```
targets = raw_targets.drop(columns=["target_weight"]).merge(
    capped_weights_df,
    on="instrument",
    how="left",
).fillna({"target_weight": 0.0})
```

por esta:

```
targets = raw_targets.drop(columns=["target_weight"]).merge(
    capped_weights_df,
    on="instrument",
    how="left",
).fillna({"target_weight": 0.0})

targets = apply_risk_manager(
    trade_date=trade_date,
    targets=targets,
    current_nlv=portfolio_value,
    previous_positions=positions,
    cfg=cfg,
)
```

Con eso la capa de riesgo actúa antes de `run_pretrade_checks()` y antes de generar órdenes.

Mejora en `execution/checks.py`

Añade una validación de sector por si el risk manager falló o faltan metadatos.

```
from market_data.metadata import get_sector_map
```

Y dentro de `run_pretrade_checks(...)`, añade al final:

```
if not targets.empty and "target_weight" in targets.columns:
    sector_map = get_sector_map(targets["instrument"].tolist())
    tmp = targets.copy()
    tmp["sector"] = tmp["instrument"].map(sector_map)
    sector_w = tmp.groupby("sector")["target_weight"].sum()
```

```

max_sector_weight = float(cfg.get("max_sector_weight", 1.0))
offenders = sector_w[sector_w > max_sector_weight + 1e-9]
if not offenders.empty:
    messages.append(
        "Sector cap violated: " +
        ", ".join(f"{sector}={weight:.4f}" for sector, weight in offenders.items())
    )

```

run_daily.py

Solo añade el seed de metadata al inicio.

```
from market_data.metadata import seed default instrument metadata
```

Y en el `__main__`:

```

if __name__ == "__main__":
    setup_logging()
    init_db()
    seed_default_instrument_metadata()

    try:
        run_pipeline()
    except Exception:
        logger.exception("Pipeline failed")
        raise

```

Mejora del resumen diario

Añade eventos de riesgo al mensaje.

```

def format_daily_summary(summary: dict) -> str:
    nlv = summary["net_liquidation_value"]
    nlv_str = "n/a" if nlv is None else f"{nlv:.2f}"

    daily_return = summary["daily_return"]
    daily_return_pct = "n/a" if daily_return is None else f"{daily_return:.2f}"

```

```

daily_pnl = summary["daily_pnl"]
daily_pnl_str = "n/a" if daily_pnl is None else f"{daily_pnl:.2f}"

target_turnover = summary["target_turnover"]
capped_turnover = summary["capped_turnover"]

tt = "n/a" if target_turnover is None else f"{target_turnover:.4f}"
ct = "n/a" if capped_turnover is None else f"{capped_turnover:.4f}"

return (
    f"*Daily trading summary*\n"
    f>Date: {summary['date']}\n"
    f>NLV: {nlv_str}\n"
    f>Daily PnL: {daily_pnl_str}\n"
    f>Daily Return: {daily_return_pct}\n"
    f>Fills: {summary['fills_count']}\n"
    f>Risk events: {summary['risk_events_count']}\n"
    f>Target Turnover: {tt}\n"
    f>Capped Turnover: {ct}"
)

```

Qué consigue esta v7

Ahora el flujo queda así:

1. Qlib genera señales
2. portfolio construye pesos
3. turnover cap recorta
4. **risk manager:**
 - bloquea por drawdown
 - recorta sectores
 - aplica cooldown
5. pre-trade checks validan
6. executor manda órdenes
7. reporting y alertas

Eso ya se parece bastante más a una arquitectura cuanti seria.

Limitaciones actuales

Todavía hay tres simplificaciones importantes:

- el sector map es manual
- el drawdown guard actúa sobre la cartera total, no por sleeve/regimen

- el cooldown usa días naturales, no sesiones de mercado

Para un MVP está bien. Para una versión más seria, tocaría usar un calendario de mercado y metadata más fiable.

Qué haría en una v8

El siguiente salto útil sería:

- **market calendar real**
- **retraining walk-forward automático**
- **riesgo por volatilidad / target vol**
- **position sizing por ATR o vol**
- **regime filter** para bajar exposición cuando el mercado empeora

La mejora más valiosa sería un **allocator por volatilidad objetivo**, porque te cambia la calidad del sizing mucho más que seguir metiendo reglas fijas.

Te puedo dejar una **v8 con target volatility, sizing por vol y market regime filter**.



adelante

Perfecto. Vamos con una **v8** que mejora el *sizing* y la exposición total, no solo los filtros.

El objetivo es dejar de repartir pesos "fijos" y pasar a esto:

1. **medir volatilidad reciente por activo**
2. **dar menos peso a lo más volátil**
3. **ajustar la exposición total de la cartera a una volatilidad objetivo**
4. **bajar riesgo si el mercado entra en un régimen débil**

La idea de arquitectura queda así:

señal → target bruto → turnover cap → risk manager → vol allocator → regime filter
→ checks → ejecución

Qué añade la v8

Tres piezas nuevas:

- target volatility portfolio
- position sizing por volatilidad
- market regime filter

Config nueva

Actualiza conf/strategy.yaml:

```
top_n: 10
min_score: null
weight_mode: inv_vol
max_weight: 0.10
hard_max_weight: 0.15
cash_buffer: 0.02

max_turnover: 0.25
hard_max_turnover: 0.60

buy_slippage_bps: 10
sell_slippage_bps: 10

adv_lookback_days: 20
min_adv_usd: 5000000

min_names: 5
benchmark_ticker: SPY

max_sector_weight: 0.35
max_portfolio_drawdown: 0.12
drawdown_action: reduce_to_cash
drawdown_cash_target: 1.00

cooldown_days_after_exit: 5

vol_lookback_days: 20
target_annual_vol: 0.18
max_gross_leverage: 1.00
min_gross_leverage: 0.20

regime_filter_enabled: true
regime_ticker: SPY
regime_ma_short: 50
regime_ma_long: 200
regime_risk_on_gross: 1.00
regime_risk_off_gross: 0.35
```


Nueva estructura

```
auto_trading_system/
├── execution/
│   ├── allocator.py
│   ├── regime.py
│   ├── risk_manager.py
│   ├── checks.py
│   └── executor.py
├── market_data/
│   ├── liquidity.py
│   ├── metadata.py
│   └── volatility.py
└── conf/
    └── strategy.yaml
```



market_data/volatility.py

Calcula volatilidad histórica simple a partir de cierres diarios.

```
from __future__ import annotations

from typing import Iterable
import numpy as np
import pandas as pd
import yfinance as yf

def fetch_realized_volatility(
    tickers: Iterable[str],
    lookback_days: int = 20,
    annualization: int = 252,
) -> pd.DataFrame:
    tickers = list(dict.fromkeys(tickers))
    if not tickers:
        return pd.DataFrame(columns=["instrument", "realized_vol"])

    period = f"{max(lookback_days * 3, 90)}d"

    df = yf.download(
        tickers=tickers,
        period=period,
        interval="1d",
        auto_adjust=True,
        progress=False,
        group_by="ticker",
```

```

        threads=True,
    )

    rows = []

    if len(tickers) == 1:
        ticker = tickers[0]
        closes = df["Close"].dropna()
        rets = closes.pct_change().dropna().tail(lookback_days)
        if not rets.empty:
            vol = float(rets.std() * np.sqrt(annualization))
            rows.append({"instrument": ticker, "realized_vol": vol})
        return pd.DataFrame(rows)

    for ticker in tickers:
        if ticker not in df:
            continue
        closes = df[ticker]["Close"].dropna()
        rets = closes.pct_change().dropna().tail(lookback_days)
        if rets.empty:
            continue
        vol = float(rets.std() * np.sqrt(annualization))
        rows.append({"instrument": ticker, "realized_vol": vol})

    return pd.DataFrame(rows)

```

execution/allocator.py

Aquí ocurre lo importante: convertir selección en pesos por volatilidad y luego ajustar la cartera a una volatilidad objetivo.

```

from __future__ import annotations

import math
import pandas as pd

def build_inverse_vol_weights(
    targets: pd.DataFrame,
    vol_df: pd.DataFrame,
    max_weight: float,
    cash_buffer: float,
    vol_floor: float = 0.05,
) -> pd.DataFrame:
    if targets.empty:
        return targets.copy()

```

```

df = targets.merge(vol_df, on="instrument", how="left").copy()
df["realized_vol"] = df["realized_vol"].fillna(vol_floor).clip(lower=v

df["inv_vol"] = 1.0 / df["realized_vol"]
df["raw_weight"] = df["inv_vol"] / df["inv_vol"].sum()

investable = max(0.0, 1.0 - cash_buffer)
df["target_weight"] = df["raw_weight"] * investable

if max_weight < 1.0:
    df["target_weight"] = df["target_weight"].clip(upper=max_weight)

total = float(df["target_weight"].sum())
if total > 0 and total < investable:
    room = max_weight - df["target_weight"]
    room = room.clip(lower=0.0)
    room_sum = float(room.sum())
    if room_sum > 0:
        df["target_weight"] += room / room_sum * (investable - tot

return df.drop(columns=["inv_vol", "raw_weight"])

def estimate_portfolio_vol(
    weights: pd.Series,
    indiv_vols: pd.Series,
) -> float:
    # Aproximación diagonal: ignora correlaciones
    # útil para un MVP; luego la puedes cambiar por covarianza completa
    return float(math.sqrt(((weights.values * indiv_vols.values) ** 2).sum

def scale_weights_to_target_vol(
    targets: pd.DataFrame,
    target_annual_vol: float,
    min_gross_leverage: float,
    max_gross_leverage: float,
    cash_buffer: float,
) -> tuple[pd.DataFrame, float | None, float]:
    df = targets.copy()

    if df.empty or "realized_vol" not in df.columns:
        return df, None, 0.0

    current_portfolio_vol = estimate_portfolio_vol(
        weights=df["target_weight"],
        indiv_vols=df["realized_vol"],
    )

    if current_portfolio_vol <= 0:
        return df, current_portfolio_vol, 0.0

```

```

scale = target_annual_vol / current_portfolio_vol
scale = min(max(scale, min_gross_leverage), max_gross_leverage)

gross_before_cash = float(df["target_weight"].sum())
df["target_weight"] = df["target_weight"] * scale

max_investable = max(0.0, 1.0 - cash_buffer)
gross_after = float(df["target_weight"].sum())
if gross_after > max_investable and gross_after > 0:
    df["target_weight"] = df["target_weight"] * (max_investable / gross_after)
    gross_after = max_investable

return df, current_portfolio_vol, gross_after

```

execution/regime.py

Filtro de régimen básico con media móvil corta/larga.

```

from __future__ import annotations

import yfinance as yf
import pandas as pd

def get_market_regime(
    ticker: str = "SPY",
    ma_short: int = 50,
    ma_long: int = 200,
) -> dict:
    period = f"{max(ma_long * 3, 400)}d"

    df = yf.download(
        tickers=[ticker],
        period=period,
        interval="1d",
        auto_adjust=True,
        progress=False,
        threads=True,
    )

    closes = df["Close"].dropna()
    if closes.empty or len(closes) < ma_long:
        raise ValueError(f"Not enough history to compute market regime for {ticker}")

    short_ma = float(closes.tail(ma_short).mean())

```

```

long_ma = float(closes.tail(ma_long).mean())
last_close = float(closes.iloc[-1])

risk_on = (last_close >= long_ma) and (short_ma >= long_ma)

return {
    "ticker": ticker,
    "last_close": last_close,
    "ma_short": short_ma,
    "ma_long": long_ma,
    "risk_on": risk_on,
    "regime": "risk_on" if risk_on else "risk_off",
}

def apply_regime_gross_scaling(
    targets: pd.DataFrame,
    regime_info: dict,
    risk_on_gross: float,
    risk_off_gross: float,
    cash_buffer: float,
) -> tuple[pd.DataFrame, float]:
    df = targets.copy()
    if df.empty:
        return df, 0.0

    desired_gross = risk_on_gross if regime_info["risk_on"] else risk_off_gross
    desired_gross = min(desired_gross, 1.0 - cash_buffer)

    current_gross = float(df["target_weight"].sum())
    if current_gross <= 0:
        return df, desired_gross

    df["target_weight"] = df["target_weight"] * (desired_gross / current_gross)
    return df, desired_gross

```

Cambios en portfolio/rebalance.py

Ahora, en vez de asignar siempre pesos iguales, soportamos `inv_vol`.

Sustituye la parte de pesos por esta:

```

from market_data.volatility import fetch_realized_volatility
from execution allocator import build_inverse_vol_weights

```

Y dentro de `build_target_positions(...)`:

```
weight_mode = cfg.get("weight_mode", "equal")
max_weight = float(cfg.get("max_weight", 1.0))
cash_buffer = float(cfg.get("cash_buffer", 0.02))

if weight_mode == "equal":
    investable = 1.0 - cash_buffer
    base_weight = min(investable / len(selected), max_weight)
    selected["target_weight"] = base_weight

    remainder = investable - selected["target_weight"].sum()
    if remainder > 0:
        selected.loc[selected.index[0], "target_weight"] += remainder

elif weight_mode == "inv_vol":
    vol_df = fetch_realized_volatility(
        selected["instrument"].tolist(),
        lookback_days=int(cfg.get("vol_lookback_days", 20)),
    )
    selected = build_inverse_vol_weights(
        targets=selected,
        vol_df=vol_df,
        max_weight=max_weight,
        cash_buffer=cash_buffer,
    )

else:
    raise NotImplementedError(f"Unsupported weight_mode: {weight_mode}")
```

Y al guardar, incluye `realized_vol` si existe:

```
keep_cols = ["date", "instrument", "target_weight", "score", "rank"]
if "realized_vol" in selected.columns:
    keep_cols.append("realized_vol")

out = selected[keep_cols].copy()
```

Cambios en `execution/risk_manager.py`

No cambia la lógica principal, pero añade una función de renormalización más flexible para convivir con la capa de volatilidad.

Sustituye `renormalize_weights(...)` por:

```
def renormalize_weights(targets: pd.DataFrame, max_total_weight: float = 0
    out = targets.copy()
    total = float(out["target_weight"].sum())
    if total <= 0:
        return out

    out["target_weight"] = out["target_weight"] * (max_total_weight / tota
    return out
```

Cambios en `execution/checks.py`

Añade validación de columnas de volatilidad y gross exposure.

```
if "realized_vol" in targets.columns:
    bad_vol = targets[targets["realized_vol"] <= 0]
    if not bad_vol.empty:
        messages.append("Non-positive realized_vol detected")

gross = float(targets["target_weight"].sum()) if "target_weight" in ta
max_gross = 1.0 - float(cfg.get("cash_buffer", 0.02))
if gross > max_gross + 1e-9:
    messages.append(f"Gross exposure {gross:.4f} exceeds max investabl
```

Cambios en `execution/executor.py`

Aquí metemos dos capas nuevas entre risk manager y checks:

- target vol scaling
- regime scaling

Añade imports:

```
from execution allocator import scale_weights_to_target_vol  
from execution regime import get_market_regime, apply_regime_gross_scaling  
from execution risk_manager import apply_risk_manager, record_risk_event
```

Y dentro de `execute_target_positions(...)` , justo después de `apply_risk_manager(...)` , añade:


```

# Target volatility scaling
targets, estimated_portfolio_vol, gross_after_vol_scaling = scale_weight(
    targets=targets,
    target_annual_vol=float(cfg.get("target_annual_vol", 0.18)),
    min_gross_leverage=float(cfg.get("min_gross_leverage", 0.20)),
    max_gross_leverage=float(cfg.get("max_gross_leverage", 1.00)),
    cash_buffer=float(cfg.get("cash_buffer", 0.02)),
)

if estimated_portfolio_vol is not None:
    record_risk_event(
        trade_date,
        "target_vol",
        f"Estimated portfolio vol={estimated_portfolio_vol:.4f}, gross
    )

# Market regime filter
if bool(cfg.get("regime_filter_enabled", False)):
    regime_info = get_market_regime(
        ticker=cfg.get("regime_ticker", "SPY"),
        ma_short=int(cfg.get("regime_ma_short", 50)),
        ma_long=int(cfg.get("regime_ma_long", 200)),
    )

    targets, regime_gross = apply_regime_gross_scaling(
        targets=targets,
        regime_info=regime_info,
        risk_on_gross=float(cfg.get("regime_risk_on_gross", 1.00)),
        risk_off_gross=float(cfg.get("regime_risk_off_gross", 0.35)),
        cash_buffer=float(cfg.get("cash_buffer", 0.02)),
    )

    record_risk_event(
        trade_date,
        "market_regime",
        f"Regime={regime_info['regime']}. ticker={regime_info['ticker']}

```

</> Python



▶ Ejecutar

Mejora en infra/db.py

Añade esta tabla para guardar snapshots de régimen, si quieres trazabilidad mejor:

```

CREATE TABLE IF NOT EXISTS regime_state (
    date TEXT PRIMARY KEY,
    ticker TEXT NOT NULL,

```

```

    regime TEXT NOT NULL,
    last_close REAL NOT NULL,
    ma_short REAL NOT NULL,
    ma_long REAL NOT NULL
);

```

Y en `execution/executor.py`, tras `get_market_regime(...)`, puedes guardar:

```

with get_conn() as conn:
    conn.execute(
        """
        INSERT OR REPLACE INTO regime_state(date, ticker, regime,
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """,
        (
            trade_date,
            regime_info["ticker"],
            regime_info["regime"],
            regime_info["last_close"],
            regime_info["ma_short"],
            regime_info["ma_long"],
        ),
    )

```

Cambios en el reporte

`reporting/daily_report.py`

Añade sección de régimen y últimos eventos de riesgo:

```

regime = pd.read_sql_query(
    "SELECT * FROM regime_state WHERE date = ?",
    conn,
    params=(trade_date,),
)

risk_events = pd.read_sql_query(
    """
    SELECT event_type, message, created_at
    FROM risk_events
    WHERE date = ?
    ORDER BY id
    """,
    conn,
    params=(trade_date,),
)

```

```
conn,  
params=(trade_date,),  
)
```

Y en el HTML:

```
<h2>Regime State</h2>  
{regime.to_html(index=False, border=0)}  
  
<h2>Risk Events</h2>  
{risk_events.to_html(index=False, border=0)}
```

Qué consigue esta v8

Ahora el sistema decide no solo **qué comprar**, sino también **cuánto riesgo total quiere llevar**.

Eso es una mejora grande porque:

- reduce concentración accidental en nombres muy volátiles
- hace la exposición total más estable
- baja riesgo automáticamente en regímenes malos
- hace el sistema menos frágil a cambios de mercado

Limitaciones importantes de esta v8

Hay tres simplificaciones claras:

1. La estimación de vol ignora correlaciones

La `estimate_portfolio_vol()` usa aproximación diagonal.

Está bien para MVP, pero una v9 sería debería usar matriz de covarianza.

2. El regime filter es muy simple

Media 50/200 funciona como guard básico, pero no captura toda la estructura del mercado.

3. El *sizing* sigue siendo long-only y diario

No hay aún budget por factor, beta neutralidad ni control de correlación.

Orden lógico de madurez

Con todo lo que llevas, ya tienes:

- señal
- selección
- liquidez
- turnover cap
- slippage
- benchmark
- reporting
- alertas
- risk manager
- target vol
- regime filter

A partir de aquí, lo más rentable ya no es añadir más reglas pequeñas, sino dar un salto en uno de estos dos sentidos:

camino A — calidad del research

- walk-forward real
- más features
- mejor validación
- ensemble de modelos

camino B — calidad del riesgo

- covarianza real
- risk parity / optimizer
- sector/factor exposure
- control de beta y correlación

La siguiente versión más valiosa sería una **v9 con matriz de covarianza, target vol real y optimizer tipo mean-variance / risk parity.**



adelante

Perfecto. Vamos con una **v9** que ya da un salto de verdad en la parte de riesgo y asignación.

Hasta ahora en la v8 tenías:

- pesos por `inverse vol`
- `target vol` aproximada
- `regime filter`
- límites por sector
- `drawdown guard`

La limitación fuerte era esta: la volatilidad de cartera se estimaba **ignorando correlaciones**. Eso hace que el sizing sea útil, pero todavía bastante tosco.

En la **v9** pasamos a una capa de asignación basada en:

- **matriz de covarianza**
- **riesgo de cartera real**
- **optimizer**
- dos modos de asignación:
 - **risk parity aproximado**
 - **mean-variance conservador**

Qué añade la v9

Tres piezas nuevas:

- **estimación de covarianza**
- **optimizer de pesos**
- **target vol de cartera real** usando covarianza

La arquitectura queda así:

signal → **target bruto** → **filtros/liquidez** → **risk manager** → **optimizer** → **target vol**
scaling → **regime filter** → **checks** → **execution**

Config nueva

Actualiza `conf/strategy.yaml`:

```
top_n: 10
min_score: null
weight_mode: optimizer
optimizer_mode: risk_parity
max_weight: 0.10
hard_max_weight: 0.15
cash_buffer: 0.02
```

```
max_turnover: 0.25
hard_max_turnover: 0.60

buy_slippage_bps: 10
sell_slippage_bps: 10

adv_lookback_days: 20
min_adv_usd: 5000000

min_names: 5
benchmark_ticker: SPY

max_sector_weight: 0.35
max_portfolio_drawdown: 0.12
drawdown_action: reduce_to_cash
drawdown_cash_target: 1.00

cooldown_days_after_exit: 5

cov_lookback_days: 60
cov_shrinkage: 0.15
target_annual_vol: 0.18
max_gross_leverage: 1.00
min_gross_leverage: 0.20

regime_filter_enabled: true
regime_ticker: SPY
regime_ma_short: 50
regime_ma_long: 200
regime_risk_on_gross: 1.00
regime_risk_off_gross: 0.35

mean_variance_risk_aversion: 4.0
optimizer_blend_with_signal: 0.30
```

Interpretación nueva

- optimizer_mode: risk_parity o mean_variance
- cov_lookback_days : ventana para covarianza
- cov_shrinkage : estabiliza matriz
- optimizer_blend_with_signal : cuánto mezclas el score del modelo con el optimizer

Estructura nueva

```
auto_trading_system/  
├── execution/  
│   ├── allocator.py  
│   ├── optimizer.py  
│   ├── regime.py  
│   ├── risk_manager.py  
│   ├── checks.py  
│   └── executor.py  
├── market_data/  
│   ├── covariance.py  
│   ├── volatility.py  
│   ├── liquidity.py  
│   └── metadata.py  
└── conf/  
    └── strategy.yaml
```



requirements.txt

Añade `scipy` , porque simplifica bastante el optimizer.

```
pyqlib  
lightgbm  
pandas  
numpy  
pyyaml  
pyarrow  
loguru  
apscheduler==3.11.0  
yfinance  
requests  
python-dotenv  
matplotlib  
scipy
```



market_data/covariance.py

Este módulo descarga precios ajustados, calcula retornos y construye la covarianza con shrinkage simple.

```
from __future__ import annotations  
  
from typing import Iterable  
import numpy as np
```

```

import pandas as pd
import yfinance as yf

def fetch_return_matrix(
    tickers: Iterable[str],
    lookback_days: int = 60,
) -> pd.DataFrame:
    tickers = list(dict.fromkeys(tickers))
    if not tickers:
        return pd.DataFrame()

    period = f"{max(lookback_days * 3, 180)}d"

    df = yf.download(
        tickers=tickers,
        period=period,
        interval="1d",
        auto_adjust=True,
        progress=False,
        group_by="ticker",
        threads=True,
    )

    price_df = pd.DataFrame()

    if len(tickers) == 1:
        ticker = tickers[0]
        closes = df["Close"].dropna()
        price_df[ticker] = closes
    else:
        for ticker in tickers:
            if ticker not in df:
                continue
            closes = df[ticker]["Close"].dropna()
            if closes.empty:
                continue
            price_df[ticker] = closes

    returns = price_df.pct_change().dropna().tail(lookback_days)
    return returns

def shrink_covariance(sample_cov: pd.DataFrame, shrinkage: float = 0.15) -
    if sample_cov.empty:
        return sample_cov

    diag = np.diag(np.diag(sample_cov.values))
    shrunk = (1.0 - shrinkage) * sample_cov.values + shrinkage * diag
    return pd.DataFrame(shrunk, index=sample_cov.index, columns=sample_cov

```



```
def build_covariance_matrix(
    tickers: Iterable[str],
    lookback_days: int = 60,
    shrinkage: float = 0.15,
    annualization: int = 252,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    returns = fetch_return_matrix(tickers, lookback_days=lookback_days)
    if returns.empty:
        return returns, pd.DataFrame()

    sample_cov = returns.cov() * annualization
    cov = shrink_covariance(sample_cov, shrinkage=shrinkage)
    return returns, cov
```

execution/optimizer.py

Aquí metemos dos modos.

```
from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.optimize import minimize

def _normalize_long_only(weights: np.ndarray, max_weight: float, total_weight: float) -> np.ndarray:
    w = np.clip(weights, 0.0, max_weight)
    s = w.sum()
    if s <= 0:
        return w
    return w * (total_weight / s)

def risk_parity_weights(
    cov: pd.DataFrame,
    max_weight: float,
    total_weight: float,
) -> pd.Series:
    assets = list(cov.columns)
    n = len(assets)

    if n == 0:
        return pd.Series(dtype=float)

    cov_m = cov.values
```

```

x0 = np.repeat(total_weight / n, n)

def portfolio_vol(w: np.ndarray) -> float:
    return float(np.sqrt(w @ cov_m @ w))

def risk_contributions(w: np.ndarray) -> np.ndarray:
    vol = portfolio_vol(w)
    if vol <= 0:
        return np.zeros_like(w)
    marginal = cov_m @ w
    return w * marginal / vol

def objective(w: np.ndarray) -> float:
    rc = risk_contributions(w)
    target = np.repeat(rc.sum() / n, n)
    return float(((rc - target) ** 2).sum())

cons = [{"type": "eq", "fun": lambda w: np.sum(w) - total_weight}]
bounds = [(0.0, max_weight) for _ in range(n)]

res = minimize(objective, x0=x0, bounds=bounds, constraints=cons, method='SLSQP')
w = x0 if not res.success else res.x
w = _normalize_long_only(w, max_weight=max_weight, total_weight=total_weight)
return pd.Series(w, index=assets)

def mean_variance_weights(
    exp_returns: pd.Series,
    cov: pd.DataFrame,
    risk_aversion: float,
    max_weight: float,
    total_weight: float,
) -> pd.Series:
    assets = list(cov.columns)
    n = len(assets)

    if n == 0:
        return pd.Series(dtype=float)

    mu = exp_returns.reindex(assets).fillna(0.0).values
    cov_m = cov.loc[assets, assets].values
    x0 = np.repeat(total_weight / n, n)

    def objective(w: np.ndarray) -> float:
        ret = mu @ w
        risk = w @ cov_m @ w
        return float(-(ret - risk_aversion * risk))

    cons = [{"type": "eq", "fun": lambda w: np.sum(w) - total_weight}]
    bounds = [(0.0, max_weight) for _ in range(n)]

    res = minimize(objective, x0=x0, bounds=bounds, constraints=cons, method='SLSQP')

```

```

w = x0 if not res.success else res.x
w = _normalize_long_only(w, max_weight=max_weight, total_weight=total_
return pd.Series(w, index=assets)

def blend_optimizer_with_signal(
    optimizer_weights: pd.Series,
    signal_scores: pd.Series,
    blend: float,
    max_weight: float,
    total_weight: float,
) -> pd.Series:
    scores = signal_scores.reindex(optimizer_weights.index).fillna(0.0)
    scores = scores - scores.min()

    if float(scores.sum()) > 0:
        signal_w = scores / scores.sum()
    else:
        signal_w = pd.Series(
            np.repeat(1.0 / len(scores), len(scores)),
            index=scores.index,
        )

    final_w = (1.0 - blend) * optimizer_weights + blend * signal_w
    final_w = final_w.clip(lower=0.0, upper=max_weight)

    s = float(final_w.sum())
    if s > 0:
        final_w = final_w * (total_weight / s)

    return final_w

```

execution/allocator.py

Sustituimos la estimación diagonal por una real con covarianza.

```

from __future__ import annotations

import math
import pandas as pd

def estimate_portfolio_vol_from_cov(
    weights: pd.Series,
    cov: pd.DataFrame,

```

```

) -> float:
    assets = [a for a in weights.index if a in cov.index]
    if not assets:
        return 0.0

    w = weights.reindex(assets).fillna(0.0).values
    cov_m = cov.loc[assets, assets].values
    return float(math.sqrt(w @ cov_m @ w))

def scale_weights_to_target_vol_from_cov(
    targets: pd.DataFrame,
    cov: pd.DataFrame,
    target_annual_vol: float,
    min_gross_leverage: float,
    max_gross_leverage: float,
    cash_buffer: float,
) -> tuple[pd.DataFrame, float | None, float]:
    df = targets.copy()
    if df.empty or cov.empty:
        return df, None, 0.0

    weights = pd.Series(df["target_weight"].values, index=df["instrument"])
    current_portfolio_vol = estimate_portfolio_vol_from_cov(weights, cov)

    if current_portfolio_vol <= 0:
        return df, current_portfolio_vol, 0.0

    scale = target_annual_vol / current_portfolio_vol
    scale = min(max(scale, min_gross_leverage), max_gross_leverage)

    df["target_weight"] = df["target_weight"] * scale

    max_investable = max(0.0, 1.0 - cash_buffer)
    gross_after = float(df["target_weight"].sum())
    if gross_after > max_investable and gross_after > 0:
        df["target_weight"] = df["target_weight"] * (max_investable / gross_after)
        gross_after = max_investable

    return df, current_portfolio_vol, gross_after

```

Cambios en portfolio/rebalance.py

Ahora puede construir pesos con optimizer.

Añade imports:

```

from market_data.covariance import build_covariance_matrix
from execution.optimizer import (
    risk_parity_weights,
    mean_variance_weights,
    blend_optimizer_with_signal,
)

```

Y dentro de `build_target_positions(...)`, sustituye la lógica de pesos por:

```

weight_mode = cfg.get("weight_mode", "equal")
max_weight = float(cfg.get("max_weight", 1.0))
cash_buffer = float(cfg.get("cash_buffer", 0.02))
investable = 1.0 - cash_buffer

if weight_mode == "equal":
    base_weight = min(investable / len(selected), max_weight)
    selected["target_weight"] = base_weight
    remainder = investable - selected["target_weight"].sum()
    if remainder > 0:
        selected.loc[selected.index[0], "target_weight"] += remainder

elif weight_mode == "inv_vol":
    vol_df = fetch_realized_volatility(
        selected["instrument"].tolist(),
        lookback_days=int(cfg.get("vol_lookback_days", 20)),
    )
    selected = build_inverse_vol_weights(
        targets=selected,
        vol_df=vol_df,
        max_weight=max_weight,
        cash_buffer=cash_buffer,
    )

elif weight_mode == "optimizer":
    _, cov = build_covariance_matrix(
        selected["instrument"].tolist(),
        lookback_days=int(cfg.get("cov_lookback_days", 60)),
        shrinkage=float(cfg.get("cov_shrinkage", 0.15)),
    )

    if cov.empty:
        raise ValueError("Covariance matrix is empty")

    mode = cfg.get("optimizer_mode", "risk_parity")

    if mode == "risk_parity":
        opt_w = risk_parity_weights(

```

```

        cov=cov,
        max_weight=max_weight,
        total_weight=investable,
    )
elif mode == "mean_variance":
    exp_returns = selected.set_index("instrument")["score"]
    opt_w = mean_variance_weights(
        exp_returns=exp_returns,
        cov=cov,
        risk_aversion=float(cfg.get("mean_variance_risk_aversion",
                                     max_weight=max_weight,
                                     total_weight=investable,
    )
else:
    raise NotImplementedError(f"Unsupported optimizer_mode: {mode}")

opt_w = blend_optimizer_with_signal(
    optimizer_weights=opt_w,
    signal_scores=selected.set_index("instrument")["score"],
    blend=float(cfg.get("optimizer_blend_with_signal", 0.30)),
    max_weight=max_weight,
    total_weight=investable,
)

selected["target_weight"] = selected["instrument"].map(opt_w).fill

else:
    raise NotImplementedError(f"Unsupported weight_mode: {weight_mode}")

```

Cambios en execution/executor.py

Cambia el target vol scaling para usar covarianza real.

Añade imports:

```

from market_data.covariance import build_covariance_matrix
from execution allocator import scale_weights_to_target_vol_from_cov

```

Y sustituye el bloque de target vol por:

```

_, cov = build_covariance_matrix(
    targets["instrument"].tolist(),
    lookback_days=int(cfg.get("cov_lookback_days", 60)),

```

```

        shrinkage=float(cfg.get("cov_shrinkage", 0.15)),
    )

    targets, estimated_portfolio_vol, gross_after_vol_scaling = scale_weights(
        targets=targets,
        cov=cov,
        target_annual_vol=float(cfg.get("target_annual_vol", 0.18)),
        min_gross_leverage=float(cfg.get("min_gross_leverage", 0.20)),
        max_gross_leverage=float(cfg.get("max_gross_leverage", 1.00)),
        cash_buffer=float(cfg.get("cash_buffer", 0.02)),
    )

```

Mejora en execution/checks.py

Añade chequeo de covarianza disponible indirectamente:

```

if len(targets) >= 2 and "instrument" in targets.columns:
    unique_names = targets["instrument"].nunique()
    if unique_names < 2:
        messages.append("Need at least 2 instruments for covariance-aware")

```

Guarda eventos del optimizer

En execution/executor.py, después de construir la covarianza y hacer scaling:

```

if not cov.empty:
    record_risk_event(
        trade_date,
        "covariance_model",
        f"Covariance assets={len(cov.columns)}, lookback={int(cfg.get(
    )

```

Y si quieres más trazabilidad, en portfolio/rebalance.py puedes registrar:

</> Python



▶ Ejecutar

Luego, cuando eliges `optimizer_mode` :

```
record_risk_event(  
    trade_date=selected["date"].iloc[0],  
    event_type="optimizer_mode",  
    message=f"Optimizer mode={mode}, blend={float(cfg.get('optimiz  
)
```

Mejora en el reporte diario

En `reporting/daily_report.py` , añade una mini tabla de exposición por sector:

```
sector_exposure = pd.read_sql_query(  
    """  
    SELECT ps.instrument, ps.weight, im.sector  
    FROM portfolio_snapshots ps  
    LEFT JOIN instrument_metadata im  
        ON ps.instrument = im.instrument  
    WHERE ps.date = ?  
    """,  
    conn,  
    params=(trade_date,),  
)
```

Y luego:

```
if not sector_exposure.empty:  
    sector_summary = (  
        sector_exposure.groupby("sector", dropna=False)["weight"]  
        .sum()  
        .reset_index()  
        .sort_values("weight", ascending=False)  
    )  
else:  
    sector_summary = pd.DataFrame(columns=["sector", "weight"])
```

En el HTML:


```
<h2>Sector Exposure</h2>
{sector_summary.to_html(index=False, border=0)}
```

Qué consigue esta v9

Ahora tu sistema hace tres cosas mejor que antes:

1. Entiende correlaciones

Dos nombres con la misma vol no son iguales si están muy correlacionados.

2. El target vol ya es más real

La volatilidad de cartera se estima con covarianza, no como suma diagonal aproximada.

3. El allocator es más serio

Puedes elegir entre:

- **risk parity** para una cartera más robusta
- **mean-variance** para aprovechar más la señal del modelo

Qué modo usaría yo primero

Para arrancar en paper trading:

- `weight_mode: optimizer`
- `optimizer_mode: risk_parity`
- `optimizer_blend_with_signal: 0.20` o `0.30`

Eso suele ser más estable que ir directamente a `mean_variance`, que es más sensible al ruido del score.

Limitaciones de la v9

Siguen quedando simplificaciones importantes:

- no hay costes dentro del optimizer
- no hay constraint de turnover dentro del solver, solo antes/después
- no hay beta neutrality
- no hay factor exposure control
- no hay covariance forecasting avanzado

Pero como MVP avanzado, ya estás bastante arriba.

Qué haría en una v10

El siguiente salto realmente potente sería uno de estos dos:

Opción A: optimizer operativo

- turnover penalty dentro del optimizer
- costes de transacción en la función objetivo
- constraint por sector dentro del solver
- cash residual explícita

Opción B: research más serio

- walk-forward automático
- ensemble de modelos
- selección dinámica de universo
- evaluación rolling por regímenes

La **v10 más valiosa** sería meter **costes y turnover directamente en el optimizer**, porque ahí ya alineas señal, riesgo y ejecución en una sola capa.

Te la desarrollo si quieres como **v10 con transaction-cost-aware optimizer y penalización por turnover**.



haz las dos opciones en la V10

Perfecto. Hagamos una **V10 combinada** con las dos líneas a la vez:

- **A. optimizer operativo**
- **B. research más serio**

La idea es que la V10 ya no solo "asigne bien", sino que además:

1. **entrene y valide mejor**
2. **optimice teniendo en cuenta coste y turnover**
3. **elija carteras más ejecutables**
4. **mida de forma más realista lo que pasará en live**

Qué cambia en la V10

La arquitectura pasa a ser esta:

research walk-forward → ensemble signal → universe selection → target bruto → cost-aware optimizer → risk manager → regime filter → checks → execution → reporting

V10: objetivos

Bloque A — Optimizer operativo

Añadimos al optimizer:

- penalización por **costes de transacción**
- penalización por **turnover**
- límites por **sector dentro del solver**
- cartera objetivo definida respecto a la **cartera actual**
- cash residual explícita

Bloque B — Research serio

Añadimos:

- **walk-forward automático**
 - **ensemble de modelos**
 - **universe dinámico**
 - evaluación rolling por periodos / regímenes
-

Estructura V10

```
auto_trading_system/  
├── conf/  
│   ├── strategy.yaml  
│   ├── research.yaml  
│   └── universe.yaml  
├── research/  
│   ├── run_walkforward.py  
│   ├── ensemble.py  
│   ├── universe_selection.py  
│   ├── evaluate_windows.py  
│   └── export_signals.py  
├── execution/  
│   ├── optimizer_v10.py  
│   ├── allocator.py  
│   ├── regime.py  
│   └── risk_manager.py
```



```
|   ├── checks.py  
|   ├── executor.py  
|   └── market_data/  
|       ├── covariance.py  
|       ├── liquidity.py  
|       ├── volatility.py  
|       └── metadata.py  
|   └── reporting/  
|       ├── daily_report.py  
|       ├── charts.py  
|       └── research_report.py  
└── infra/  
    ├── db.py  
    ├── logging_conf.py  
    ├── notifications.py  
    └── paths.py
```

Configuración nueva

conf/strategy.yaml

```
top_n: 15  
min_score: null  
weight_mode: optimizer  
optimizer_mode: cost_aware_mvo  
  
max_weight: 0.10  
hard_max_weight: 0.15  
cash_buffer: 0.02  
  
max_turnover: 0.25  
hard_max_turnover: 0.60  
turnover_penalty: 3.0  
transaction_cost_penalty: 2.0  
  
buy_slippage_bps: 10  
sell_slippage_bps: 10  
commission_bps: 10  
  
adv_lookback_days: 20  
min_adv_usd: 5000000  
  
min_names: 5  
benchmark_ticker: SPY  
  
max_sector_weight: 0.35
```

```
max_portfolio_drawdown: 0.12
drawdown_action: reduce_to_cash
drawdown_cash_target: 1.00

cooldown_days_after_exit: 5

cov_lookback_days: 60
cov_shrinkage: 0.15
target_annual_vol: 0.18
max_gross_leverage: 1.00
min_gross_leverage: 0.20

regime_filter_enabled: true
regime_ticker: SPY
regime_ma_short: 50
regime_ma_long: 200
regime_risk_on_gross: 1.00
regime_risk_off_gross: 0.35

mean_variance_risk_aversion: 4.0
optimizer_blend_with_signal: 0.20
```

conf/research.yaml

```
models:
  - name: lgbm
    weight: 0.5
  - name: xgboost
    weight: 0.3
  - name: linear
    weight: 0.2

walkforward:
  train_years: 3
  valid_months: 6
  test_months: 1
  step_months: 1

evaluation:
  benchmark: SPY
  regimes:
    enabled: true
    ticker: SPY
    ma_short: 50
    ma_long: 200
```

conf/universe.yaml

```
universe_mode: dynamic
max_names_pre_filter: 100
min_price: 5.0
min_adv_usd: 5000000
include_etfs: false
```

A. Optimizer operativo

Idea principal

Hasta la V9 el optimizer decidía pesos sin "sentir" del todo:

- lo que cuesta moverse
- lo lejos que está de la cartera actual
- cuánto turnover implica

En la V10 optimizamos sobre **pesos finales** y no sobre una cartera abstracta.

La función objetivo será algo así:

maximizar

- retorno esperado
- menos riesgo
- menos coste de transacción
- menos turnover

Forma de la función objetivo

Para pesos w :

- $\mu^\top w$ = alpha esperado
- $w^\top \Sigma w$ = riesgo
- $|w - w_{\text{prev}}|$ = turnover
- coste por activo = $tc_i * |w_i - w_{\text{prev}_i}|$

Objetivo:

$$\max \mu^\top w - \lambda_r w^\top \Sigma w - \lambda_t \sum_i |w_i - w_i^{\text{prev}}| - \lambda_c \sum_i c_i |w_i - w_i^{\text{prev}}|$$

Como solver práctico para un MVP, lo resolvemos con `scipy.optimize.minimize`.

execution/optimizer_v10.py

```

from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.optimize import minimize

def build_prev_weights(
    instruments: list[str],
    current_positions: dict[str, int],
    prices: dict[str, float],
    portfolio_value: float,
) -> pd.Series:
    rows = {}
    if portfolio_value <= 0:
        return pd.Series(0.0, index=instruments)

    for instrument in instruments:
        qty = int(current_positions.get(instrument, 0))
        px = float(prices.get(instrument, 0.0))
        rows[instrument] = 0.0 if px <= 0 else (qty * px) / portfolio_value

    return pd.Series(rows).reindex(instruments).fillna(0.0)

def build_transaction_cost_vector(
    instruments: list[str],
    adv_df: pd.DataFrame | None,
    commission_bps: float,
    buy_slippage_bps: float,
    sell_slippage_bps: float,
    min_adv_usd: float = 1e6,
) -> pd.Series:
    base = (commission_bps + 0.5 * (buy_slippage_bps + sell_slippage_bps))
    costs = pd.Series(base, index=instruments, dtype=float)

    if adv_df is None or adv_df.empty:
        return costs

    tmp = adv_df.set_index("instrument")["adv_usd"].reindex(instruments)
    liquidity_penalty = (min_adv_usd / tmp.clip(lower=min_adv_usd)).fillna(1)
    return costs * liquidity_penalty

def sector_constraint_builder(
    sector_map: dict[str, str],
    instruments: list[str],

```

```

max_sector_weight: float,
):
    sectors = sorted(set(sector_map.values()))

    constraints = []
    for sector in sectors:
        idx = [i for i, inst in enumerate(instruments) if sector_map.get(i) == sector]
        if not idx:
            continue

        constraints.append({
            "type": "ineq",
            "fun": lambda w, idx=idx: max_sector_weight - np.sum(w[idx]),
        })
    return constraints

def cost_aware_mean_variance_weights(
    exp_returns: pd.Series,
    cov: pd.DataFrame,
    prev_weights: pd.Series,
    tc_vector: pd.Series,
    risk_aversion: float,
    turnover_penalty: float,
    transaction_cost_penalty: float,
    max_weight: float,
    total_weight: float,
    sector_map: dict[str, str] | None = None,
    max_sector_weight: float | None = None,
) -> pd.Series:
    assets = list(cov.columns)
    n = len(assets)

    if n == 0:
        return pd.Series(dtype=float)

    mu = exp_returns.reindex(assets).fillna(0.0).values
    cov_m = cov.loc[assets, assets].values
    w_prev = prev_weights.reindex(assets).fillna(0.0).values
    tc = tc_vector.reindex(assets).fillna(tc_vector.mean() if len(tc_vector) > 0 else 0)

    x0 = np.repeat(total_weight / n, n)

    def objective(w: np.ndarray) -> float:
        ret = mu @ w
        risk = w @ cov_m @ w
        turnover = np.abs(w - w_prev).sum()
        costs = (tc * np.abs(w - w_prev)).sum()

        return float(-(ret - risk_aversion * risk - turnover_penalty * turnover - transaction_cost_penalty * costs))

    cons = [{"type": "eq", "fun": lambda w: np.sum(w) - total_weight}]

```



```

if sector_map is not None and max_sector_weight is not None:
    cons.extend(sector_constraint_builder(sector_map, assets, max_sector_weight))

bounds = [(0.0, max_weight) for _ in range(n)]

res = minimize(objective, x0=x0, method="SLSQP", bounds=bounds, constraints=cons)
w = x0 if not res.success else res.x
w = np.clip(w, 0.0, max_weight)

s = w.sum()
if s > 0:
    w = w * (total_weight / s)

return pd.Series(w, index=assets)

def post_optimize_turnover_soft_cap(
    target_weights: pd.Series,
    prev_weights: pd.Series,
    max_turnover: float,
) -> pd.Series:
    delta = target_weights.sub(prev_weights, fill_value=0.0)
    turnover = float(np.abs(delta).sum() / 2.0)

    if turnover <= max_turnover or turnover == 0:
        return target_weights

    scale = max_turnover / turnover
    capped = prev_weights + delta * scale

    total = float(capped.sum())
    if total > 0:
        capped = capped * (float(target_weights.sum()) / total)

    return capped

```

Integración del optimizer operativo

En portfolio/rebalance.py

Añade imports:

```

from market_data.covariance import build_covariance_matrix
from market_data.liquidity import fetch_average_dollar_volume
from market_data.metadata import get_sector_map

```

```
from execution.optimizer_v10 import (
    build_prev_weights,
    build_transaction_cost_vector,
    cost_aware_mean_variance_weights,
    post_optimize_turnover_soft_cap,
)
```

Y en el modo `optimizer`, usa esta lógica:

```
elif weight_mode == "optimizer":
    trade_date = str(selected["date"].iloc[0])

    _, cov = build_covariance_matrix(
        selected["instrument"].tolist(),
        lookback_days=int(cfg.get("cov_lookback_days", 60)),
        shrinkage=float(cfg.get("cov_shrinkage", 0.15)),
    )
    if cov.empty:
        raise ValueError("Covariance matrix is empty")

    from execution.paper_broker import PaperBroker
    broker = PaperBroker()
    positions = broker.get_positions()

    price_map = {inst: 1.0 for inst in selected["instrument"].tolist()}
    portfolio_value = 1.0

    prev_weights = build_prev_weights(
        instruments=selected["instrument"].tolist(),
        current_positions=positions,
        prices=price_map,
        portfolio_value=portfolio_value,
    )

    adv_df = fetch_average_dollar_volume(
        selected["instrument"].tolist(),
        lookback_days=int(cfg.get("adv_lookback_days", 20)),
    )

    tc_vector = build_transaction_cost_vector(
        instruments=selected["instrument"].tolist(),
        adv_df=adv_df,
        commission_bps=float(cfg.get("commission_bps", 10)),
        buy_slippage_bps=float(cfg.get("buy_slippage_bps", 10)),
        sell_slippage_bps=float(cfg.get("sell_slippage_bps", 10)),
        min_adv_usd=float(cfg.get("min_adv_usd", 1e6)),
    )

    sector_map = get_sector_map(selected["instrument"].tolist())
```

```

opt_w = cost_aware_mean_variance_weights(
    exp_returns=selected.set_index("instrument")["score"],
    cov=cov,
    prev_weights=prev_weights,
    tc_vector=tc_vector,
    risk_aversion=float(cfg.get("mean_variance_risk_aversion", 4.0)),
    turnover_penalty=float(cfg.get("turnover_penalty", 3.0)),
    transaction_cost_penalty=float(cfg.get("transaction_cost_penalty", 1.0)),
    max_weight=max_weight,
    total_weight=investable,
    sector_map=sector_map,
    max_sector_weight=float(cfg.get("max_sector_weight", 1.0)),
)

opt_w = post_optimize_turnover_soft_cap(
    target_weights=opt_w,
    prev_weights=prev_weights.reindex(opt_w.index).fillna(0.0),
    max_turnover=float(cfg.get("max_turnover", 1.0)),
)

selected["target_weight"] = selected["instrument"].map(opt_w).fillna(0.0)

```

Nota importante

En esa integración he dejado `price_map = 1.0` como placeholder, pero en una versión real conviene pasar precios actuales de mercado o snapshots recientes. La idea arquitectónica es correcta; ese detalle hay que afinarlo en código real.

B. Research serio

Ahora vamos a la otra mitad.

1. Walk-forward automático

En vez de un único train/valid/test fijo, generamos ventanas rodantes:

- entrenas en 3 años
- validas en 6 meses
- test en 1 mes
- avanzas 1 mes y repites

Eso te da una evaluación mucho más honesta.

`research/run_walkforward.py`

```

from __future__ import annotations

from dataclasses import dataclass
import pandas as pd

@dataclass
class WFWindow:
    train_start: str
    train_end: str
    valid_start: str
    valid_end: str
    test_start: str
    test_end: str

def generate_walkforward_windows(
    start_date: str,
    end_date: str,
    train_years: int = 3,
    valid_months: int = 6,
    test_months: int = 1,
    step_months: int = 1,
) -> list[WFWindow]:
    start = pd.Timestamp(start_date)
    end = pd.Timestamp(end_date)

    windows = []
    cursor = start + pd.DateOffset(years=train_years, months=valid_months)

    while True:
        train_end = cursor - pd.DateOffset(months=valid_months)
        train_start = train_end - pd.DateOffset(years=train_years)

        valid_start = train_end
        valid_end = valid_start + pd.DateOffset(months=valid_months)

        test_start = valid_end
        test_end = test_start + pd.DateOffset(months=test_months)

        if test_end > end:
            break

        windows.append(
            WFWindow(
                train_start=train_start.strftime("%Y-%m-%d"),
                train_end=train_end.strftime("%Y-%m-%d"),
                valid_start=valid_start.strftime("%Y-%m-%d"),
                valid_end=valid_end.strftime("%Y-%m-%d"),
                test_start=test_start.strftime("%Y-%m-%d"),

```

```

        test_end=test_end.strftime("%Y-%m-%d"),
    )
)
cursor += pd.DateOffset(months=step_months)

return windows

```

2. Ensemble de modelos

No dependes de un solo modelo; mezclas varias señales.

research/ensemble.py

```

from __future__ import annotations

import pandas as pd

def zscore_by_date(df: pd.DataFrame, score_col: str = "score") -> pd.DataFrame:
    out = df.copy()
    out[score_col] = out.groupby("date")[score_col].transform(
        lambda s: (s - s.mean()) / (s.std() if s.std() > 0 else 1.0)
    )
    return out

def combine_model_signals(model_dfs: list[tuple[str, pd.DataFrame, float]]) -> pd.DataFrame:
    parts = []

    for model_name, df, weight in model_dfs:
        tmp = df.copy()
        tmp = zscore_by_date(tmp, score_col="score")
        tmp["weighted_score"] = tmp[score_col] * weight
        tmp["model_name"] = model_name
        parts.append(tmp[["date", "instrument", "weighted_score", "model_name"]])

    merged = pd.concat(parts, ignore_index=True)
    out = (
        merged.groupby(["date", "instrument"], as_index=False)[score_col]
        .sum()
        .rename(columns={"weighted_score": score_col})
    )
    return out

```

3. Universo dinámico

No trabajas siempre con el mismo universo estático. Cada día o mes construyes uno invertible.

research/universe_selection.py

```
from __future__ import annotations

import pandas as pd
from market_data.liquidity import fetch_average_dollar_volume

def select_dynamic_universe(
    instruments: list[str],
    max_names_pre_filter: int = 100,
    min_adv_usd: float = 5_000_000,
    adv_lookback_days: int = 20,
) -> pd.DataFrame:
    adv = fetch_average_dollar_volume(instruments, lookback_days=adv_lookb
    if adv.empty:
        return pd.DataFrame(columns=["instrument", "adv_usd"])

    adv = adv[adv["adv_usd"] >= min_adv_usd].copy()
    adv = adv.sort_values("adv_usd", ascending=False).head(max_names_pre_f
    return adv.reset_index(drop=True)
```

4. Evaluación rolling por regímenes

Quieres saber si tu señal funciona igual en:

- mercado risk-on
- mercado risk-off

research/evaluate_windows.py

```
from __future__ import annotations

import pandas as pd
```

```
def summarize_window_results(results_df: pd.DataFrame) -> pd.DataFrame:
    # espera columnas: window_id, period_return, sharpe, max_drawdown, reg
    if results_df.empty:
        return pd.DataFrame()

    agg = (
        results_df.groupby("regime", as_index=False)
        .agg(
            mean_return=("period_return", "mean"),
            mean_sharpe=("sharpe", "mean"),
            mean_max_dd=("max_drawdown", "mean"),
            n_windows=("window_id", "nunique"),
        )
    )
    return agg
```

Cómo queda el flujo research de la V10

Paso 1

Generas ventanas walk-forward.

Paso 2

Para cada ventana:

- entrenas varios modelos
- validas
- generas predicciones en test

Paso 3

Combinas predicciones con ensemble.

Paso 4

Mides resultados por ventana y por régimen.

Paso 5

Solo despliegas combinaciones que aguanten bien rolling OOS.

research/export_signals.py en la V10

En vez de leer una sola predicción, exporta el ensemble final.

```
from __future__ import annotations

import pandas as pd
from research.ensemble import combine_model_signals

def export_ensemble_signals(model_signal_map: list[tuple[str, pd.DataFrame]]):
    signals = combine_model_signals(model_signal_map)
    signals = signals.sort_values(["date", "score"], ascending=[True, False])
    signals.to_parquet(out_path, index=False)
    return out_path
```

Mejora en la base de datos

Añade tablas research:

```
CREATE TABLE IF NOT EXISTS walkforward_runs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    window_id TEXT NOT NULL,
    train_start TEXT NOT NULL,
    train_end TEXT NOT NULL,
    valid_start TEXT NOT NULL,
    valid_end TEXT NOT NULL,
    test_start TEXT NOT NULL,
    test_end TEXT NOT NULL,
    model_name TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS research_metrics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    window_id TEXT NOT NULL,
    model_name TEXT NOT NULL,
    regime TEXT,
    period_return REAL,
    sharpe REAL,
    max_drawdown REAL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```


Reporte research V10

reporting/research_report.py

</> Python



Ejecutar

```
from __future__ import annotations

from pathlib import Path
import pandas as pd

from infra.db import get_conn
from infra.paths import REPORTS_DIR

def generate_research_report() -> Path:
    with get_conn() as conn:
        wf = pd.read_sql_query("SELECT * FROM walkforward_runs ORDER BY id")
        metrics = pd.read_sql_query("SELECT * FROM research_metrics ORDER BY id")

    out_path = REPORTS_DIR / "research_report.html"

    html = f"""
    <html>
    <head><meta charset="utf-8"><title>Research Report</title></head>
    <body>
        <h1>Walkforward Runs</h1>
        {wf.to_html(index=False, border=0)}
        <h1>Research Metrics</h1>
        {metrics.to_html(index=False, border=0)}
    </body>
    </html>
    """

    out_path.write_text(html, encoding="utf-8")
    return out_path
```

Qué consigue esta V10 combinada

En ejecución

Ahora el optimizer ya no busca solo pesos "bonitos". Busca pesos que:

- respeten riesgo
- respeten sector
- penalicen mover demasiado la cartera
- penalicen costes

- tengan más probabilidad de sobrevivir al paso a live

En research

Ahora la señal ya no depende de:

- una sola muestra temporal
- un solo modelo
- un único universo estático

Eso mejora mucho la robustez.

Orden de prioridad para implementarla

Yo lo montaría en este orden:

Fase 1

Research:

- walk-forward
- ensemble
- métricas rolling

Fase 2

Execution:

- optimizer con turnover y coste
- sector constraints dentro del solver

Fase 3

Unión:

- exportar ensemble como señal única
 - pasarla por optimizer operativo
 - comparar contra V9
-

Qué revisaría para validar la V10

No mires solo retorno. Mira sobre todo:

- turnover medio antes y después de penalización
- coste implícito estimado
- estabilidad de pesos entre días
- performance OOS por ventana

- performance en risk-on vs risk-off
 - concentración sectorial efectiva
 - tracking del benchmark
 - cuántas veces el optimizer colapsa a soluciones casi iguales
-

Limitaciones de esta V10

Todavía quedan cosas por afinar:

- el optimizer usa `scipy`, no un solver convexo más sofisticado
- el modelo de coste es simple
- el turnover penalty es L1 aproximado vía optimización general
- no hay constraint explícita de beta
- no hay factor model

Pero como arquitectura, esta ya es una **V10 seria**.

Qué sería una V11 lógica

La siguiente iteración valiosa sería:

- **factor model** en vez de cov simple
- **beta neutrality opcional**
- **coste no lineal por liquidez**
- **solver más robusto**
- **selección automática de modelo según ventana/regime**

La mejora con más impacto práctico sería meter **modelo de costes no lineal + factor risk model**.

